



Sensors and Instrumentation

Final Project Report

Smart Weather Station

Lebanese University – Faculty of Engineering III

Department of Mechanical Engineering

Academic Year 2025-2026

Submitted to:

Dr. Mohamad Aoude

Prepared by:

Imane Hojairy 6663

Malak Abou Daher 6569

Mohamad Al Hajjar 6886

Serena Al Hajjar 6885



Abstract

This project presents the design and implementation of an Arduino-based smart weather station capable of monitoring several environmental and physical quantities in real time. The system measures temperature, humidity, light intensity, atmospheric pressure, altitude, relative air quality, water-flow rate, total water volume, and weight. An Arduino Mega is used as the main controller and receives data from the DHT22, BH1750, BMP280, MQ135, HX711 with load cell, and water-flow sensor.

The sensor readings are processed and displayed on a 16×2 LCD. A push button allows the user to move between the different measurements and system functions. The project also includes automatic responses. A servo motor controls a model clothesline according to the measured light intensity, while a buzzer warns the user when the temperature exceeds a predefined threshold. Filtering, averaging, calibration, and hysteresis were used to improve the stability and reliability of the measurements and outputs.

The system also provides minimum and maximum temperature tracking, sensor-status monitoring, pressure-trend analysis, bright-time calculation, and a basic local weather estimation. In addition, the measurements are sent through the Serial Monitor in CSV format for later analysis and visualization. The completed prototype demonstrates how several sensing functions, real-time display, data logging, and automatic control can be combined in one low-cost embedded monitoring system.

Table of Contents

1. Introduction.
2. Problem Statement.
3. System Architecture.
4. Sensor Selection and Justification.
5. Hardware Design and Schematics.
6. Software Design and Firmware Architecture.
7. Calibration Procedures and Results.
8. Experimental Results and Data Analysis.
9. Measurement Uncertainty and Error Analysis.
10. Engineering Constraints Discussion.
11. Conclusion and Future Work.
12. References.

1. Introduction

Environmental conditions such as temperature, humidity, light, pressure, and air quality can affect comfort, safety, and many daily activities. For this reason, monitoring these conditions in real time can be useful, especially when the system can also react automatically when something changes.

In this project, we designed and built a smart weather station using an Arduino Mega. The system combines several sensors in one setup and measures temperature, humidity, light intensity, atmospheric pressure, estimated altitude, relative air-quality changes, weight, water-flow rate, and total water volume.

The measurements are collected and processed by the Arduino, then shown on a 16×2 LCD. Because the screen is small, a push button is used to move from one reading to another. The system also sends the data through the Serial Monitor in CSV format, so the results can be saved and studied later.

The project also includes two automatic actions. A servo motor controls a small clothesline depending on the light level, while a buzzer gives a warning when the temperature becomes too high. In addition, the program tracks the minimum and maximum temperature, calculates the pressure trend, bright-time duration, and total water volume, and gives a simple local weather estimation.

The final prototype shows how different sensors, data processing, display, logging, and automatic control can be combined in one practical and low-cost embedded system.

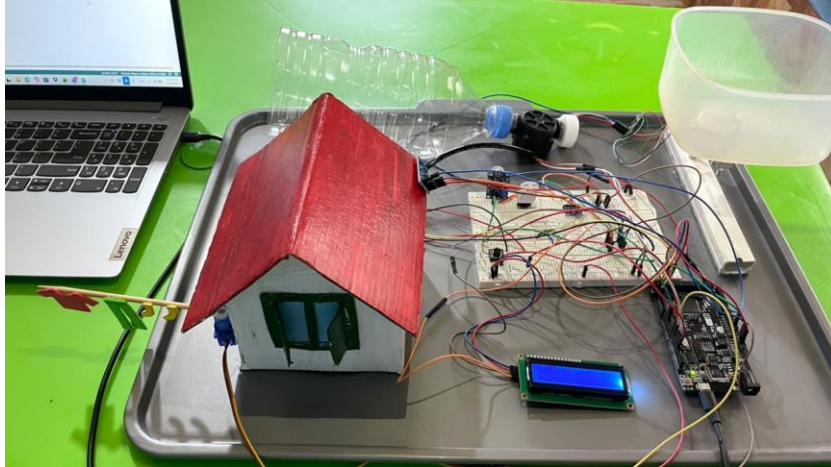


Fig. 1. *Final assembled Arduino-based smart weather station.*

2. Problem Statement

Environmental conditions can change throughout the day and may affect comfort, safety, water use, equipment performance, and indoor spaces. However, many low-cost monitoring devices only measure one or two variables, which gives only a limited view of the surroundings. A simple thermometer, for example, can show temperature, but it cannot provide information about humidity, light intensity, air-quality variation, atmospheric pressure, water flow, or applied weight. It also cannot react automatically when a certain condition becomes abnormal.

For this reason, the aim of this project was to build a low-cost smart weather station that could monitor several environmental and physical quantities at the same time. The system includes the five required sensing types: optical, force, flow, temperature, and humidity. Pressure and relative air-quality measurements were also added to make the station more useful and to give a more complete description of the surrounding environment.

The main challenge was not simply reading each sensor separately, but making all of them work together in one system. The sensors use different types of signals and communication methods. The DHT22 uses a digital signal, the BH1750 and BMP280 communicate through the I²C bus, the MQ135 provides an analog reading, the HX711 uses separate data and clock lines, and the flow sensor generates pulses that must be counted using an interrupt. All of these signals had to be handled by the Arduino Mega in the same program without causing conflicts.

Once the sensors were connected, another difficulty was obtaining stable and useful readings. Measurements can be affected by electrical noise, changes in the environment, sensor response time, mechanical vibration, load position, water-pressure variation, air bubbles, and the limited power supplied through the laptop USB port. To reduce these effects, the program applies validity checks, filtering, averaging, calibration factors, and threshold conditions. Temperature, humidity, light, and flow readings are filtered, while the load-cell value is obtained from the average of five readings.

The system also needed to present the information in a clear way. Since the 16×2 LCD is too small to show all measurements at once, the data were divided into several screens. A push button allows the user to move between them and view temperature, humidity, light intensity, clothesline status, relative air quality, pressure, altitude, weight, flow rate, total water volume, minimum and maximum temperature, system status, forecast, pressure trend, and bright-time duration. The same readings are also sent through the Serial Monitor in CSV format so they can be saved and analyzed later.

In addition to monitoring, the system was designed to respond automatically. A buzzer warns the user when the temperature rises above the selected limit, while a servo motor controls a small model clothesline according to the measured light intensity. Two different light thresholds are used for opening and closing so that the servo does not keep moving when the light level is close to the switching point.

Some parts of the original design also had to be changed during testing. A rain sensor was first used to control the clothesline, but it stopped giving reliable readings. An SD-card module was also tested for local data storage, but it did not initialize consistently. To improve the reliability of the final system, the rain sensor was replaced by the BH1750 light sensor, and Serial CSV logging was used instead of SD-card storage.

The final challenge was therefore to build one complete and reliable system that could read several sensors continuously, process the measurements, display the

results clearly, react to abnormal conditions, control the servo motor and buzzer automatically, and record the data for later analysis.

3. System Architecture

The smart weather station is built around an Arduino Mega, which acts as the main controller of the system. All sensors, the LCD, the push button, the servo motor, and the buzzer are connected to the Arduino. It receives the sensor readings, processes the data, displays the results, sends the measurements to the Serial Monitor in CSV format, and controls the outputs according to the programmed conditions.

3.1 Inputs

The system inputs are the sensors used to measure the different environmental and physical quantities.

The **DHT22 sensor** measures ambient temperature and relative humidity. These values are displayed on the LCD and are also used for the high-temperature warning, minimum and maximum temperature tracking, and the basic weather forecast.

The **BH1750 sensor** measures light intensity in lux. Its reading is displayed on the LCD and is used to control the servo-operated clothesline. It is also used to calculate the total bright-time duration and contributes to the forecast logic.

The **BMP280 sensor** measures atmospheric pressure and estimates altitude using a reference sea-level pressure. Its measurements are also used to determine whether the pressure is rising, falling, or remaining stable.

The **MQ135 sensor** provides an analog value that changes when smoke or certain gases are present around it. In this project, it is used as a relative air-quality indicator rather than as a calibrated air-quality index. Readings below 400 are classified as good, values from 400 to 599 as medium, and values of 600 or more as poor.

The **HX711 module and load cell** are used to measure weight. When a load is applied, the load cell produces a very small electrical signal. The HX711 amplifies this signal and converts it into a digital value that can be read by the Arduino.

The **water-flow sensor** produces digital pulses as water passes through it. The Arduino counts these pulses and converts them into a flow rate in liters per minute. The same readings are used to calculate the total volume of water that has passed through the sensor.

3.2 Processing

The Arduino Mega performs all data acquisition, calculations, filtering, monitoring, and control decisions.

The program continuously reads all sensors and checks whether the obtained values are valid. Unrealistic temperature or humidity readings are rejected before being used. Temperature, humidity, light, and flow values are filtered to reduce sudden fluctuations, while the load-cell result is obtained by averaging five readings.

The flow-sensor pulses are counted using an interrupt. The pulse frequency is converted into flow rate using the calibration factor, and the total water volume is calculated by accumulating the amount of water passing during each time interval.

The program also tracks the minimum and maximum valid temperature measured since startup. The first valid temperature initializes both values. The minimum changes only when a lower temperature is detected, while the maximum changes only when a higher temperature is measured.

Atmospheric-pressure readings are compared every 30 seconds. If the new pressure is higher by more than 0.2 hPa, the pressure trend is classified as rising. If it decreases by more than 0.2 hPa, it is classified as falling. Otherwise, it is considered stable.

A basic local forecast is generated using temperature, humidity, light intensity, atmospheric pressure, and pressure trend. Depending on the conditions, the system may display Stable, Humid, Clear, Hot and Dry, Rain Possible, or Sensor Error. This forecast is rule-based and does not use internet weather data.

The bright-time function counts the total duration during which the light intensity remains above the selected threshold. The counter starts when the Arduino is powered on and resets when the system is restarted.

The Arduino also checks the operating condition of the main sensors. The system-status screen displays either **All Sensors OK** or **Check Sensors**.

Finally, the Arduino controls the servo motor and buzzer. The servo responds to light intensity, while the buzzer responds to high temperature.

3.3 Outputs

The main system outputs are the LCD, Serial CSV data, servo motor, buzzer, and the visual warning shown on the LCD.

The **16×2 LCD** displays the direct sensor measurements and the calculated information. Since it has limited space, the information is divided into eleven screens. A push button allows the user to move between the screens.

The LCD displays:

1. Temperature and humidity
2. Light intensity and clothesline status
3. Air-quality reading and classification
4. Pressure and altitude
5. Weight
6. Water-flow rate and total water volume
7. Minimum and maximum temperature
8. System status
9. Weather forecast

10. Pressure trend

11. Bright-time duration

The **Serial Monitor** receives all measurements in CSV format. Each row includes elapsed time, temperature, humidity, light intensity, air-quality reading, pressure, altitude, weight, flow rate, total volume, forecast, pressure trend, bright time, and clothesline status. This format allows the data to be exported to Excel for plotting and analysis.

The **servo motor** controls the model clothesline. It moves to the open position when the filtered light intensity reaches 105 lux or more and moves to the closed position when the light decreases to 95 lux or less. Between these two values, the servo keeps its previous position. This hysteresis prevents repeated opening and closing when the light fluctuates near the threshold.

The **buzzer** is used as a high-temperature warning. The Arduino continuously compares the filtered temperature with the selected threshold. When the temperature exceeds the limit, the buzzer turns on and off repeatedly. At the same time, the LCD displays the message **HIGH TEMP!**, providing both an audible and visual warning. When the temperature returns below the threshold, the buzzer is switched off automatically.

3.4 Architecture Figure

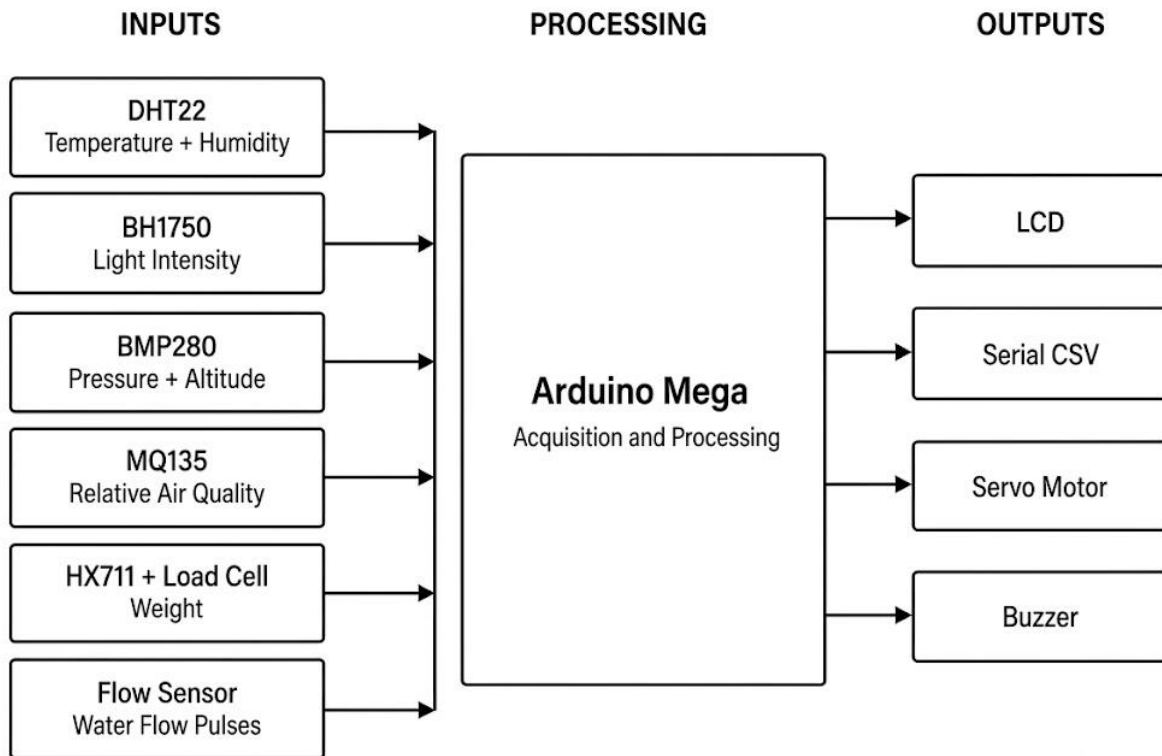


Fig. 2. *Overall block diagram and data flow of the smart weather station.*

4. Sensor Selection and Justification

The components used in the smart weather station were selected according to the required sensing modalities, the information needed for the weather report, compatibility with the Arduino board, and ease of integration. Each component has

a specific role in the system, from measuring environmental conditions to displaying data and producing automatic responses.

4.1 DHT22 Temperature and Humidity Sensor

The DHT22 sensor was selected to measure the surrounding temperature and relative humidity. It satisfies two of the mandatory sensing modalities required in the project: temperature sensing and humidity sensing.

The sensor provides its measurements through a digital signal, which makes it simple to connect directly to the Arduino. In this project, its data pin is connected to digital pin D2.

The measured temperature and humidity values are shown continuously on the LCD. The temperature reading is also used to activate the buzzer when the selected temperature limit is exceeded. In addition, the program records the minimum and maximum temperature values measured during operation. Both temperature and humidity are also considered when generating the simplified weather forecast.

One limitation of the DHT22 is its relatively slow response time compared with more advanced sensors. It may also occasionally return an invalid reading, especially immediately after startup or if measurements are requested too frequently. Therefore, the program checks the readings before using them.

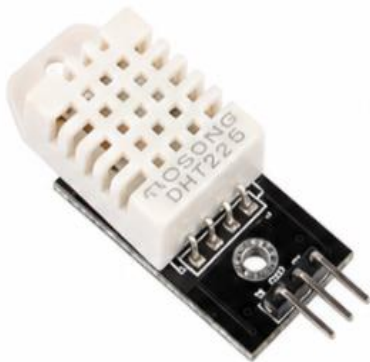


Fig. 3. *DHT22 temperature & humidity sensor*

4.2 BH1750 Light Sensor

The BH1750 was selected to measure the surrounding light intensity directly in lux. Unlike basic light-dependent resistors, the BH1750 provides a calibrated digital light value that can be easily interpreted.

The sensor communicates with the Arduino using the I²C communication protocol. It shares the SDA and SCL lines with the other I²C components in the project.

The BH1750 controls the automatic movement of the servo motor, which represents a clothesline. When the light intensity reaches or exceeds 105 lux, the servo moves to the open position. When the light intensity falls to 95 lux or below, the servo returns to the closed position.

Using two different thresholds creates a 10-lux gap between opening and closing. This gap prevents the servo from moving repeatedly when the light level fluctuates around a single threshold.

The light reading is also used in the weather forecast and in the bright-time calculation, which records how long the station has been exposed to sufficiently bright conditions.

A limitation of the BH1750 is that the reading can be affected by the sensor position, shadows, nearby artificial lighting, and the direction from which the light reaches the sensor.



Fig. 4. BH1750 light sensor

4.3 HX711 Module and Load Cell

The load cell and HX711 module were selected to provide the required force-sensing modality by measuring applied weight.

When a load is placed on the load cell, the strain gauges inside it produce a very small electrical voltage. This signal is too weak to be read directly by the Arduino.

The HX711 module is therefore used to amplify the load-cell signal and convert it into a digital value.

The HX711 communicates with the Arduino using two digital connections: the data pin, known as DT, and the clock pin, known as SCK. In this project, DT is connected to D11 and SCK is connected to D10.

The scale is tared automatically during startup so that the initial unloaded condition is treated as zero. A calibration factor of -400000 is used to convert the raw readings into kilograms. To improve stability, the displayed weight is calculated using the average of five consecutive readings.

The load-cell system is sensitive to vibration, movement, uneven surfaces, and the position of the applied load. Placing the weight away from the correct loading point may produce inaccurate results.

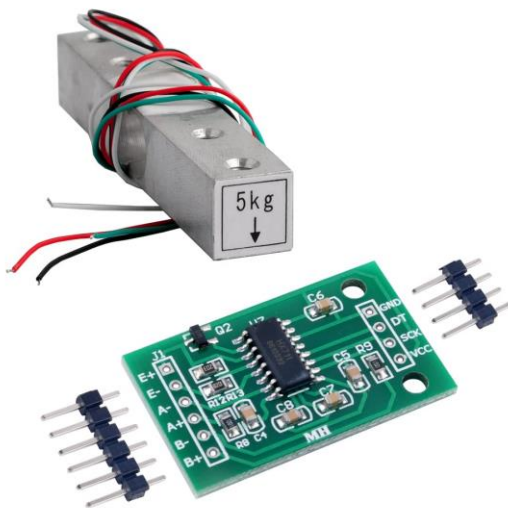


Fig. 5. Load cell connected to HX711

4.4 Water-Flow Sensor

The water-flow sensor was selected to satisfy the required flow-sensing modality. It measures the rate at which water passes through the sensor.

Inside the sensor, a small rotor rotates as water flows through it. The rotation generates electrical pulses. The pulse frequency increases as the water-flow rate increases.

The signal wire is connected to the Arduino interrupt pin D18. Using an interrupt allows the Arduino to count the pulses accurately while continuing to read the other sensors.

The counted pulse frequency is converted into a flow rate using a calibration factor of 7.6. The instantaneous flow rate is displayed in liters per minute, written as L/min. The program also accumulates the measured flow over time to calculate the total water volume in liters.

The readings may be affected by air bubbles, unstable water pressure, changes in pipe diameter, turbulence, and the orientation of the sensor. For accurate results, the sensor should be calibrated using a known water volume.



Fig. 6. Water-flow sensor

4.5 BMP280 Pressure Sensor

The BMP280 was selected to measure atmospheric pressure and estimate altitude. These measurements provide additional environmental information and make the project more similar to a practical weather-monitoring system.

The sensor communicates with the Arduino using the I²C communication protocol. It shares the same SDA and SCL connections used by the LCD and BH1750.

Atmospheric pressure is displayed in hectopascals, while the altitude is estimated in meters. The pressure measurements are also used to analyze whether the atmospheric pressure is rising, falling, or remaining stable. This pressure trend contributes to the simplified weather forecast.

The altitude value is not measured directly. It is calculated from atmospheric pressure using an assumed sea-level reference pressure. Therefore, the estimated altitude may be inaccurate if the reference pressure does not match the actual local weather conditions.

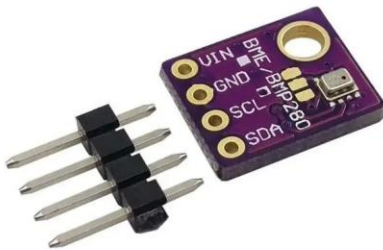


Fig. 7. BMP280 pressure sensor

4.6 MQ135 Gas Sensor

The MQ135 was selected to detect relative changes in the presence of smoke and different gases in the surrounding air. It provides a simple indication of whether the air conditions are improving or becoming worse.

The sensor output is connected to the analog input pin A1. The Arduino reads the output as a raw analog value.

For easier interpretation, the program divides the readings into three categories:

- Below 400: Good
- From 400 to 599: Medium
- 600 or more: Poor

The raw value and its classification are displayed on the LCD.

The MQ135 does not provide a certified Air Quality Index value. Its output is affected by temperature, humidity, sensor age, supply voltage, and the gases present in the environment. It also requires a warm-up period before its readings become relatively stable. Therefore, it is used only to observe relative changes and not as a professional air-quality measurement device.



Fig. 8. MQ135 gas sensor

4.7 LCD Display

A 16×2 liquid crystal display was selected to present the measured values and system information directly to the user without requiring a computer.

The LCD communicates with the Arduino using an I²C interface. This reduces the number of required connections because only the SDA and SCL communication lines are needed in addition to power.

The display contains 11 different screens. These screens show information such as temperature, humidity, light intensity, pressure, altitude, air-quality level, weight,

water-flow rate, total water volume, temperature limits, pressure trend, forecast information, bright time, and servo position.

Because the display can show only two lines at a time, the push button is used to move between the available screens.



Fig. 9. *LCD Display*

4.8 Push Button

The push button was added to allow the user to navigate manually between the 11 LCD screens.

It is connected to digital pin D7 and configured using the Arduino `INPUT_PULLUP` setting. This activates the Arduino's internal pull-up resistor, eliminating the need for an additional external resistor.

With this configuration, the input normally reads HIGH and changes to LOW when the button is pressed.

A software debounce delay is included in the program. This prevents one physical press from being detected as several presses because of the small electrical fluctuations that occur when the button contacts close.



Fig. 10. *Push Button*

4.9 Servo Motor

The servo motor was selected as an output device to demonstrate an automatic response to the measured environmental conditions. In the project, it represents a motorized clothesline or a similar light-controlled mechanism.

The servo signal wire is connected to digital pin D8.

Two main positions are used:

- Closed position: 0°
- Open position: 90°

The servo is controlled automatically using the light intensity measured by the BH1750 sensor. It opens when the light level reaches 105 lux and closes when the light level decreases to 95 lux.

A limitation of the servo is that it can draw a relatively high current, especially during movement or when mechanically blocked. Since the project is powered through the laptop USB connection, the servo should operate without carrying a heavy mechanical load.



Fig. 11. *Servo Motor*

4.10 Buzzer

The buzzer was selected to provide an audible warning when the measured temperature becomes higher than the selected safety threshold.

It is connected to digital pin D12 and is controlled by the temperature value received from the DHT22 sensor.

Instead of producing a continuous sound, the buzzer operates intermittently. It turns on and off at regular intervals while the temperature remains above the threshold. This makes the warning noticeable without producing an uninterrupted and uncomfortable tone.

The buzzer is used only as a simple warning device and does not indicate the exact temperature. The actual measured value and warning condition are shown on the LCD.



Fig. 12. *Buzzer*

5. Hardware Design and Schematics

This section describes the hardware used in the project and explains how the sensors, display, and actuators were connected to the Arduino Mega. The complete prototype is powered from the laptop through the USB cable. The Arduino then supplies power to the connected modules, while all components share the same ground connection.

The hardware design was arranged so that the sensors could work together without pin conflicts. The DHT22, MQ135, HX711, flow sensor, push button, servo motor, and buzzer use separate Arduino pins, while the LCD, BH1750, and BMP280 share the same I²C communication lines.

5.1 Main Components

The final system includes the components listed in Table 1.

Component	Function in the project
Arduino Mega	Reads the sensors, processes the data, and controls the outputs
DHT22	Measures temperature and humidity
BH1750	Measures light intensity
BMP280	Measures atmospheric pressure and estimates altitude
MQ135	Detects relative changes in air quality
HX711	Amplifies and converts the load-cell signal
Load cell	Measures applied weight
Water-flow sensor	Measures flow rate and total water volume
16×2 LCD	Displays the measurements and system information
Push button	Changes between the LCD screens
Servo motor	Controls the model clothesline
Buzzer	Gives a high-temperature warning
Breadboard and jumper wires	Connect the components during testing and operation

Table 1. *Main components used in the final prototype.*

5.2 Arduino Pin Connections

The final pin connections are shown in Table 2.

Component	Signal pin	Arduino Mega pin
DHT22	DATA	D2
Push button	Signal	D7
Servo motor	Signal	D8
HX711	SCK	D10
HX711	DT	D11
Buzzer	Signal	D12
Water-flow sensor	Signal	D18
MQ135	Analog output	A1
LCD	SDA	D20
LCD	SCL	D21
BH1750	SDA	D20
BH1750	SCL	D21
BMP280	SDA	D20
BMP280	SCL	D21

Table 2. *Pin connections between the components and the Arduino Mega.*

The LCD, BH1750, and BMP280 are connected to the same I²C bus. Their SDA pins are connected to Arduino pin 20, while their SCL pins are connected to pin 21. They can share the same lines because each device has a different I²C address.

The flow sensor is connected to digital pin 18 because this pin supports hardware interrupts. This allows the Arduino to count the flow pulses while continuing to read the other sensors and update the display.

All ground terminals are connected together. This common ground provides the same voltage reference for every sensor and output device and helps the Arduino read the signals correctly.

5.3 Power Supply

The Arduino Mega is powered from the laptop using a USB cable. The Arduino then provides 5 V and ground to the breadboard and connected modules.

Most of the sensors require only a small amount of current. However, the MQ135 includes an internal heater, and the servo motor draws additional current when it moves. Since the whole system is powered from the laptop USB port, the available current is limited.

For this prototype, the servo was connected to a light mechanism and did not carry a heavy load. This helped reduce sudden current demand and kept the system stable during testing.

In a future version, the servo motor could be powered from a separate regulated 5 V supply. The external supply ground would still need to be connected to the Arduino ground.

5.4 Sensor Connections

The DHT22 data line is connected to digital pin 2. The sensor sends both temperature and humidity readings through the same digital connection.

The MQ135 analog output is connected to pin A1. The Arduino reads the output as a raw analog value and uses it to show relative changes in air quality.

The load cell is connected first to the HX711 module. The HX711 then sends the processed measurement to the Arduino through the DT and SCK pins. The load cell is fixed under a small platform so that objects can be placed on top for weight measurement.

The flow sensor signal wire is connected to digital pin 18. When water passes through the sensor, the internal rotor generates pulses. The Arduino counts these pulses and uses them to calculate flow rate and accumulated water volume.

The BH1750 and BMP280 are connected through the shared I²C lines. The BH1750 provides the light reading, while the BMP280 provides pressure and estimated altitude.

5.5 Output Connections

The 16×2 LCD is connected through the same I²C bus used by the BH1750 and BMP280. This reduces the number of Arduino pins needed for the display.

The push button is connected to digital pin 7 and uses the Arduino internal pull-up resistor. When the button is pressed, the input changes from HIGH to LOW, allowing the user to move to the next LCD screen.

The servo signal wire is connected to digital pin 8. The servo is mounted inside the model structure and represents the opening and closing of the clothesline.

The buzzer is connected to digital pin 12. It produces an intermittent warning when the temperature becomes higher than the selected limit.

5.6 Signal Conditioning

The load-cell signal is too small to be measured directly by the Arduino, so the HX711 module is used to amplify it and convert it into digital data.

The MQ135 module already includes the basic circuit needed to provide an analog output. Its value can therefore be read directly by the Arduino.

The flow sensor produces digital pulses and does not require analog amplification. However, the pulses are counted using an interrupt to avoid missing them while the Arduino is performing other tasks.

Software processing is also used to improve the readings. Temperature, humidity, light, and flow values are filtered, while five load-cell readings are averaged before the weight is displayed.

5.7 Wiring Schematic

Figure 13. shows the complete wiring schematic of the final prototype. The Arduino Mega is connected to the DHT22, BH1750, BMP280, MQ135, HX711 with load cell, and water-flow sensor. The same circuit also includes the LCD, push button, servo motor, and buzzer. The LCD, BH1750, and BMP280 share the I²C communication lines, while the flow sensor is connected to an interrupt-capable pin. All components share the same ground connection and are powered through the Arduino supply.

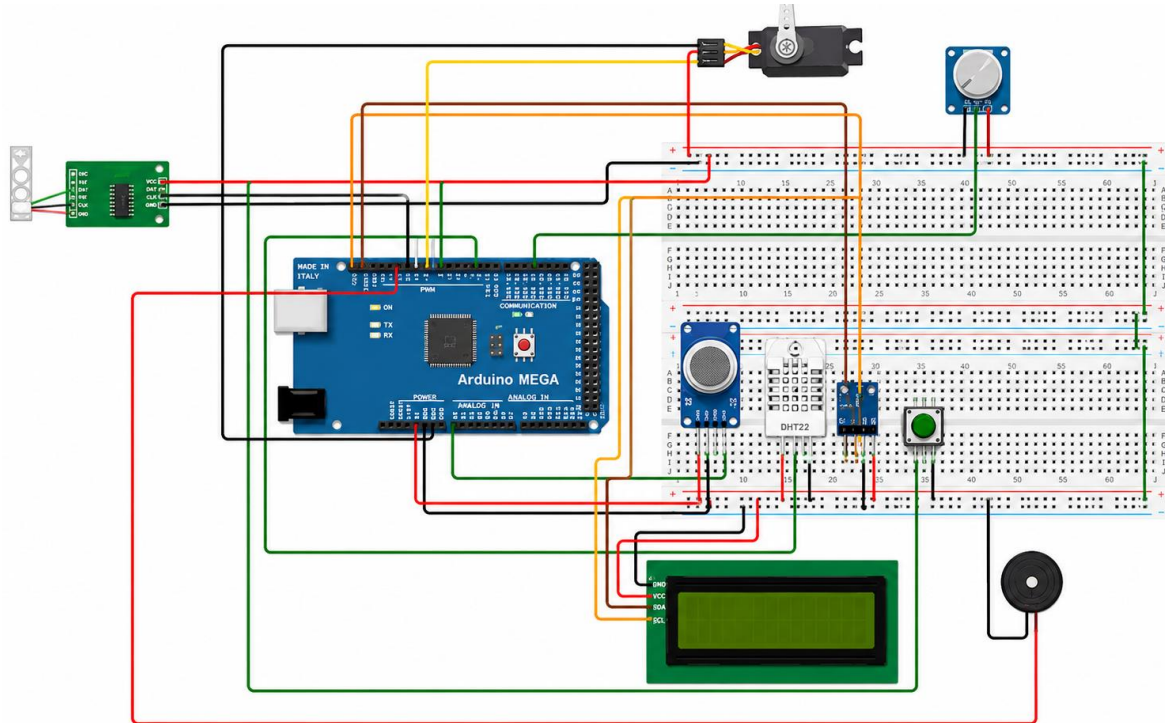


Fig. 13. *Complete wiring schematic of the smart weather station.*

5.8 Design Evolution and Problems Encountered

The final system was developed through several stages of testing, integration, and redesign. Some components included in the original design did not provide sufficiently reliable results when operating as part of the complete system. These components were tested and troubleshooted before being replaced or removed.

The rain sensor and SD-card module were not included in the final implementation. However, their development and testing are important parts of the project because they demonstrate the engineering process used to identify problems, evaluate alternatives, and improve system reliability.

5.8.1 Original Rain-Sensor Concept

The original design used a rain-detection module as the main input for controlling the servo-operated clothesline. The purpose of this feature was to protect clothes from rain by closing the clothesline automatically when water was detected.

The intended control logic was:

- Rain detected → clothesline closes
- No rain detected → clothesline opens

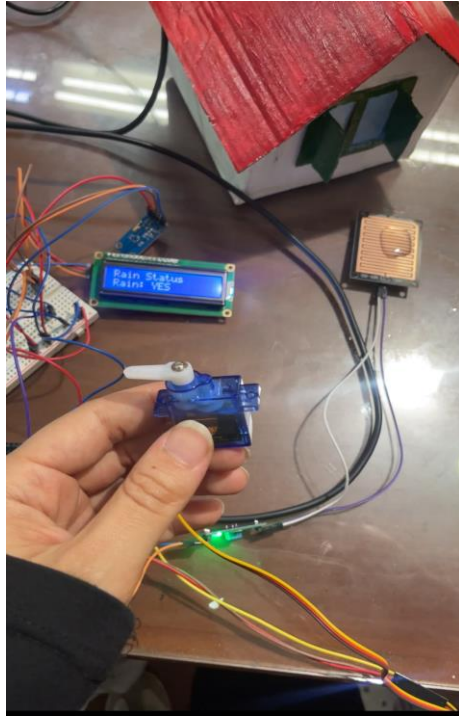


Fig. 14. Original rain-sensor module tested for automatic clothesline control

(Rain Status:YES)

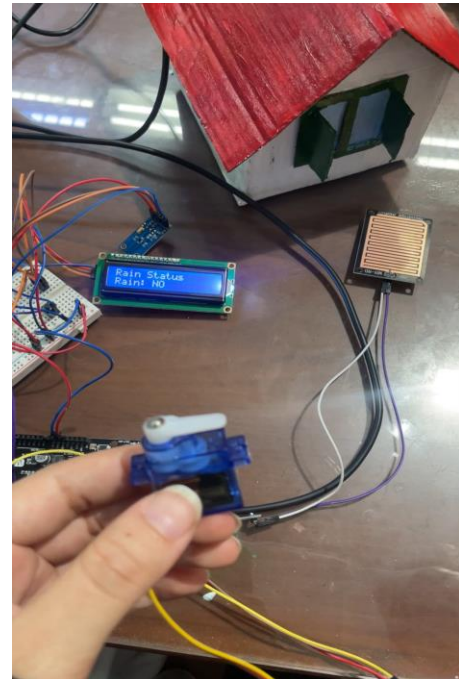


Fig. 15. Original rain-sensor module tested for automatic clothesline control

(Rain Status:NO)

The rain module was tested during the early development stages and initially provided an appropriate method for detecting wet conditions. However, its readings became unstable during the integration of the complete system.

During integration testing, the rain-detection module became unreliable and could no longer provide stable measurements. To ensure dependable automatic operation, the module was excluded from the final design.

Possible variations in the readings may have been caused by moisture remaining on the sensing surface, corrosion, sensitivity adjustment, unstable electrical connections, or contamination of the sensor plate. Since an unreliable reading could cause incorrect and repeated clothesline movement, the rain sensor was not used in the final control system.

5.8.2 Replacement Using the BH1750

After the rain sensor was removed, the BH1750 light sensor was selected as the input for controlling the servo motor. This allowed the clothesline mechanism to operate automatically according to the measured light intensity.

The final control logic was defined as follows:

- Light intensity greater than or equal to 105 lux → clothesline opens
- Light intensity less than or equal to 95 lux → clothesline closes
- Light intensity between 95 lux and 105 lux → previous servo position is maintained

The open position of the servo is 90°, while the closed position is 0°.

Using two separate thresholds introduced hysteresis into the control system. Hysteresis means that the condition required to open the clothesline is different from the condition required to close it.

The 10-lux gap between the opening and closing thresholds prevents repeated servo movement when the light level fluctuates near a single threshold. For example, after the clothesline opens at 105 lux, it remains open until the light level decreases to 95 lux or below. Values between these limits do not change the current servo position.

This redesign provided more stable automatic control and made use of the BH1750 sensor, which was already included in the system for light measurement, bright-time calculation, and weather forecasting.

5.8.3 SD-Card Data-Logging Attempt

An SD-card module was included in the original design to provide local data logging. The intended purpose was to save the measurements directly to a memory card so that the recorded environmental data could be reviewed and analyzed later. During development, several troubleshooting procedures were performed, including:

- Testing different chip-select pins
- Checking the electrical wiring and power connections

- Repeating the card-initialization process
- Testing memory-card compatibility
- Checking the communication between the Arduino and the SD-card module

Despite these tests, the SD-card module could not be initialized consistently during system-level operation. Its behavior was not sufficiently reliable for inclusion in the final implementation.

Because local SD-card logging could not be made reliable during system-level integration, the final implementation used CSV output through the Serial Monitor. This provided a stable method for exporting the measurements to Excel for analysis.

The Arduino sends the sensor measurements as comma-separated values. These values can be copied from the Serial Monitor, saved as a CSV file, and opened in Microsoft Excel. This method allows the collected data to be organized into columns and used to generate tables, calculations, and graphs.

Although Serial Monitor logging requires the Arduino to remain connected to the laptop, it was more stable and dependable than the tested SD-card implementation. Therefore, it was selected as the final method for exporting the system measurements.

The removal of the rain sensor and SD-card module should not be considered hidden failures. These changes demonstrate practical engineering decision-making. Unreliable components were identified through testing, alternative solutions were evaluated, and the final design was modified to improve system stability and overall performance.

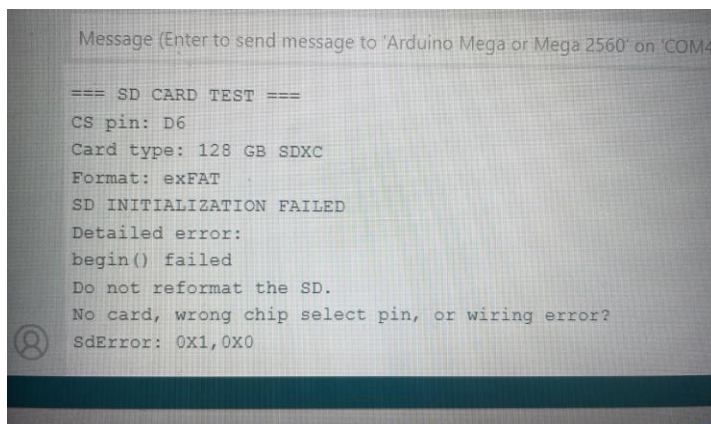


Fig. 16. Serial Monitor output showing the unsuccessful initialization of the SD-card module during testing.

6. Software Design and Firmware Architecture

The software was developed using a modular testing and integration approach. During the first stage of development, each sensor and subsystem was tested using a separate Arduino program. Individual test programs were created for the DHT22, BMP280, BH1750, MQ135, water-flow sensor, HX711 and load cell, LCD display, and logging functions. Testing the components separately made it easier to verify the wiring, check the required libraries, observe the sensor readings, and identify problems before integrating the complete system.

After the individual component tests were completed, the required functions were combined into one final program named `SmartWeatherStation.ino`. The final firmware reads all sensors, filters selected measurements, calculates additional values, controls the servo motor and buzzer, manages the LCD screens, handles push-button navigation, and exports the measurements in CSV format.

The separate component programs are not imported into the final code. They were used only for individual testing and troubleshooting. The tested functions were later integrated directly into `SmartWeatherStation.ino`.

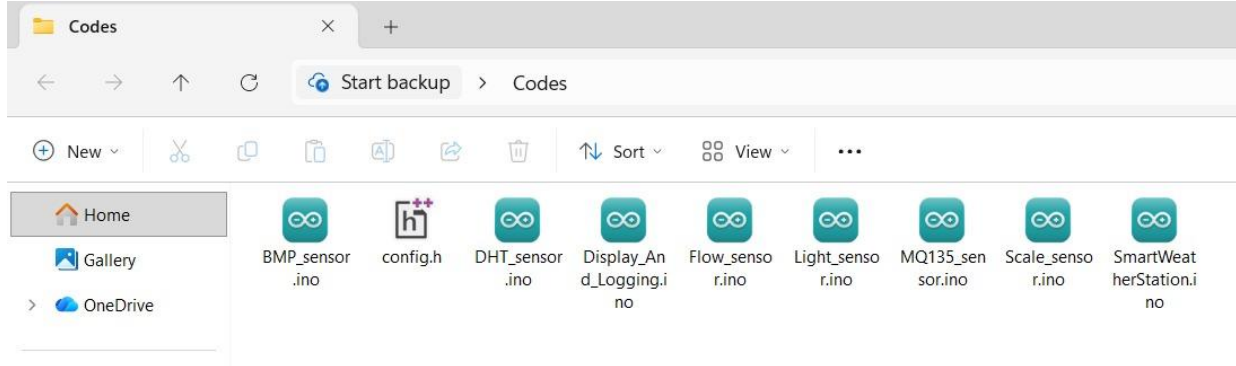


Fig. 17. *Project Software Files*

Figure 17 shows the software files created during the project. The files `BMP_sensor.ino`, `DHT_sensor.ino`, `Flow_sensor.ino`, `Light_sensor.ino`, `MQ135_sensor.ino`, `Scale_sensor.ino`, and `Display_And_Logging.ino` were used to test the main sensors and software functions independently.

The final file, `SmartWeatherStation.ino`, combines the required functions into the complete firmware.

The project also contains a `config.h` file. This file stores the two calibration constants used by the system: the HX711 load-cell calibration factor and the water-flow sensor calibration factor. Keeping these constants in a separate configuration file makes recalibration easier because they can be modified without changing the main program logic.

6.1 Libraries, Pin Definitions, and Configuration

The firmware begins by including the libraries required by the connected components. The `Wire` library supports I²C communication, while `LiquidCrystal_I2C` controls the 16×2 LCD. The DHT22 uses the `DHT` library, the BH1750 uses the `BH1750` library, and the BMP280 uses the `Adafruit_BMP280` library. The `Servo` library controls the clothesline mechanism, while the `HX711` library reads the load-cell interface.

The program also includes the local `config.h` file, which contains the load-cell and water-flow calibration constants.

The pin definitions identify the Arduino Mega pins used by each component. The program also defines the temperature-warning threshold, light thresholds, servo positions, timing intervals, and button-debounce period.

The final temperature-warning threshold is 30°C. The servo opens when the measured light reaches 105 lux and closes when the light falls to 95 lux. The LCD is refreshed every second, CSV data is produced every 60 seconds, pressure trend is evaluated every 30 seconds, and button debounce is set to 250 milliseconds.

Listing 1: Libraries, configuration file, pin assignments, and system constants.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>
#include <BH1750.h>
#include <Servo.h>
#include <Adafruit_BMP280.h>
#include <HX711.h>
#include "config.h"
#define DHT_PIN 2
#define DHT_TYPE DHT22
#define BUTTON_PIN 7
#define SERVO_PIN 8
#define HX711_SCK 10
#define HX711_DT 11
#define BUZZER_PIN 12
#define FLOW_PIN 18
#define MQ135_PIN A1
const float TEMP_LIMIT = 30.0;
const float OPEN_LIGHT = 105.0;
const float CLOSE_LIGHT = 95.0;
const int SERVO_OPEN = 90;
const int SERVO_CLOSED = 0;
const unsigned long LCD_INTERVAL = 1000;
const unsigned long CSV_INTERVAL = 60000;
const unsigned long TREND_INTERVAL = 30000;
const unsigned long BUTTON_DELAY = 250;
```


Listing 2: Calibration constants stored in config.h.

```
#ifndef CONFIG_H
#define CONFIG_H

const float SCALE_FACTOR = -400000.0;

const float FLOW_FACTOR = 7.6;

#endif
```

6.2 Setup Function

The `setup()` function executes once when the Arduino is powered on or restarted. Its purpose is to configure the communication interfaces, initialize the sensors and output devices, tare the load cell, attach the water-flow interrupt, initialize the timing variables, and print the CSV heading row.

Serial communication is initialized at 9600 baud. It is used for monitoring, debugging, and exporting the measurements. The push button and water-flow sensor use `INPUT_PULLUP`, while the buzzer is configured as an output and initially switched off.

The I²C bus is initialized using `Wire.begin()`. The LCD, BH1750, and BMP280 share this communication bus.

The LCD is initialized, its backlight is enabled, and the startup message Smart Weather Station is displayed for two seconds.

The DHT22 is started using `dht.begin()`. The BH1750 is initialized in continuous high-resolution mode. The BMP280 is first tested using address 0x76, and if necessary, the program also attempts address 0x77.

The servo motor is attached to D8 and moved to the closed position of 0°. The HX711 is initialized using its DT and SCK pins. If the module is ready, the load-cell calibration factor is applied. The LCD asks the user to remove all weight before the scale is tared using 15 readings.

The water-flow interrupt is attached to D18 and is triggered by the falling edge of each pulse. Finally, the timing variables are initialized and the CSV column headings are printed.

<pre> void setup() { Serial.begin(9600); pinMode(BUTTON_PIN, INPUT_PULLUP); pinMode(BUZZER_PIN, OUTPUT); pinMode(FLOW_PIN, INPUT_PULLUP); digitalWrite(BUZZER_PIN, LOW); Wire.begin(); lcd.init(); lcd.backlight(); lcd.setCursor(0, 0); lcd.print("Smart Weather"); lcd.setCursor(0, 1); lcd.print("Station"); delay(2000); dht.begin(); lightOk = lightSensor.begin(BH1750::CONTINUOUS_HI GH_RES_MODE); if (bmp.begin(0x76) bmp.begin(0x77)) bmpOk = true; lineServo.attach(SERVO_PIN); lineServo.write(SERVO_CLOSED); scale.begin(HX711_DT, HX711_SCK); delay(500); </pre>	<pre> if (scale.is_ready()) { scaleOk = true; scale.set_scale(SCALE_FACTOR); lcd.clear(); lcd.setCursor(0, 0); lcd.print("Remove weight"); lcd.setCursor(0, 1); lcd.print("Taring..."); delay(2000); scale.tare(15); } attachInterrupt(digitalPinToInterrupt(FLOW_PIN), countFlowPulse, FALLING); lastFlow = millis(); lastCsv = millis(); lastTrend = millis(); lastBright = millis(); lcd.clear(); } </pre>
--	---

Listing 3: Main initialization operations in the final `setup()` function.

Listing 4: CSV heading row printed during initialization.

```
Serial.println(  
  
  "Time_min,Temp_C,Hum_percent,Light_lux,"  
  "AirQuality,Pressure_hPa,Altitude_m,"  
  "Weight_kg,Flow_L_min,Total_L,"  
  "Weather,PressureTrend,BrightTime_min,"  
  "Clothesline"  
);
```

6.3 Main Loop

The `loop()` function runs continuously while the Arduino remains powered. It calls the main sensing, processing, control, display, and logging functions in a fixed sequence.

The firmware first reads the sensors and updates the water-flow calculation. It then filters the selected measurements and updates the minimum and maximum temperature.

The program evaluates the pressure trend, generates the local forecast, and updates the accumulated bright time. It then controls the servo motor and buzzer and checks the push button.

The LCD is refreshed every second. CSV data is sent through the Serial connection once every 60 seconds.

The program uses `millis()` for timing rather than long delays. This allows the sensors, controls, and button to continue operating while the program waits for the next LCD or CSV update.

```
void loop()
{
    readSensors();
    updateFlow();
    filterValues();
    updateTemperatureHistory();
    updatePressureTrend();
    updateWeather();
    updateBrightTime();
    controlServo();
    controlBuzzer();
    readButton();
    if (millis() - lastLcd >= LCD_INTERVAL)
    {
        lastLcd = millis();
        showScreen();
    }
    if (millis() - lastCsv >= CSV_INTERVAL)
    {
        lastCsv = millis();
        printCsv();
    }
}
```

Listing 5: Main execution sequence of the final firmware.

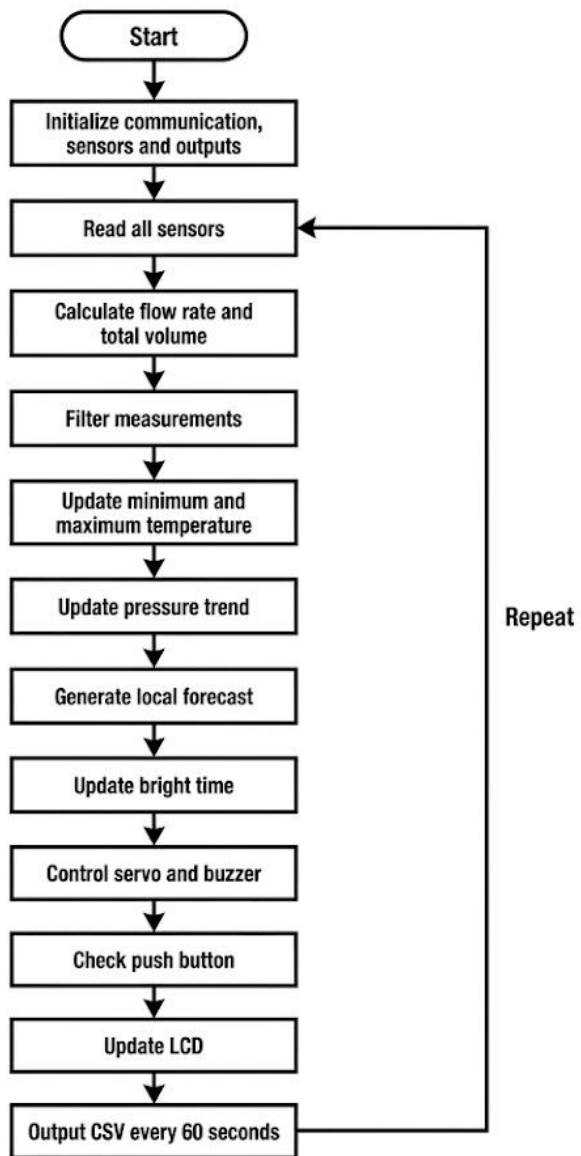


Fig. 18. *Main execution flow of the Smart Weather Station firmware.*

6.4 Sensor-Reading Function

The `readSensors()` function collects the direct measurements from the sensors.

The DHT22 provides temperature and humidity. The MQ135 raw analog output is read from A1 using `analogRead()`.

If the BH1750 was initialized successfully, the light level is read directly in lux.

If the BMP280 is available, pressure is read in pascals and divided by 100 to convert it to hectopascals. Altitude is estimated using a reference sea-level pressure of 1013.25 hPa.

When the HX711 is ready, the load-cell reading is obtained using `get_units(5)`. This function averages five readings. Very small weight values between -0.005 kg and 0.005 kg are treated as zero to reduce zero-point fluctuations.

```
void loop()
{
  readSensors(void readSensors()
  {
    temp = dht.readTemperature();
    hum = dht.readHumidity();
    airQuality = analogRead(MQ135_PIN);
    if (lightOk)
      lightLux = lightSensor.readLightLevel();
    if (bmpOk)
    {
      pressure = bmp.readPressure() / 100.0;
      altitude = bmp.readAltitude(1013.25);
    }
    if (scale.is_ready())
```

```
      if (scale.is_ready())
      {
        scaleOk = true;
        weightKg = scale.get_units(5);
        if (weightKg > -0.005 && weightKg <
0.005)
          weightKg = 0;
      }
    }
```

Listing 6: Final sensor-reading function.

6.5 Measurement Filtering

Filtering is applied to temperature, humidity, light intensity, and water-flow rate to reduce short-term fluctuations.

Temperature readings are accepted only when they are valid and between 5°C and 60°C. The filtered value is calculated using 80% of the previous value and 20% of the new measurement.

Humidity readings are accepted only when they are between 0% and 100%. Humidity uses the same 80% previous and 20% new smoothing ratio.

The BH1750 light measurement uses faster smoothing because it directly controls the servo. The filtered value uses 40% of the previous reading and 60% of the new reading.

The water-flow value uses 80% of the previous value and 20% of the current flow rate. The load-cell measurement is averaged separately using `get_units(5)`.

Listing 7: Final measurement-filtering code.

```
void filterValues()
{
    if (!isnan(temp) && temp > 5 && temp <
60)
    {
        if (isnan(filteredTemp))
            filteredTemp = temp;
        else
            filteredTemp = 0.8 * filteredTemp + 0.2
* temp;
    }
    if (!isnan(hum) && hum >= 0 && hum <=
100)
    {
        if (isnan(filteredHum))
            filteredHum = hum;
```

```
    else
        filteredHum = 0.8 * filteredHum + 0.2 *
hum;
    }
    if (lightLux >= 0)
    {
        if (filteredLight < 0)
            filteredLight = lightLux;
        else
            filteredLight =
0.4 * filteredLight + 0.6 * lightLux;
    }
    filteredFlow =
0.8 * filteredFlow + 0.2 * flowRate;
}
```

The load-cell averaging is performed using:

```
weightKg = scale.get_units(5);
```

6.6 Water-Flow Calculation

The flow sensor produces electrical pulses while water passes through it. Each falling-edge pulse activates the interrupt routine and increments the pulse counter.

The pulse variable is declared using **volatile** because it is modified inside an interrupt service routine.

Listing 8: Water-flow pulse-counting interrupt.

```
volatile unsigned long flowPulses = 0;

void countFlowPulse()
{
    flowPulses++;
}
```

The **updateFlow()** function performs the flow calculation approximately once per second.

The program temporarily disables interrupts while it copies and resets the pulse count. This prevents the value from changing while it is being processed.

The pulse frequency is calculated using the number of pulses and the elapsed time. The result is divided by the calibration factor of 7.6 to calculate the flow rate in liters per minute.

The total volume is calculated by multiplying the flow rate by the elapsed time and accumulating the result in liters.


```

void updateFlow()
{
    unsigned long now = millis();
    if (now - lastFlow >= 1000)
    {
        unsigned long elapsed = now - lastFlow;
        noInterrupts();
        unsigned long pulses = flowPulses;
        flowPulses = 0;
        interrupts();
        float frequency =
            pulses * 1000.0 / elapsed;
        flowRate = frequency / FLOW_FACTOR;
        totalLiters +=
            flowRate * elapsed / 60000.0;
        lastFlow = now;
    }
}

```

Listing 9: Flow-rate and accumulated-volume calculation.

6.7 Servo Hysteresis Control

The servo represents the automatic clothesline mechanism. It is controlled using the filtered light measurement from the BH1750.

When the clothesline is closed and the light reaches or exceeds 105 lux, the servo moves to 90° and the clothesline status becomes open.

When the clothesline is open and the light falls to 95 lux or below, the servo moves to 0° and the status becomes closed.

When the light remains between 95 lux and 105 lux, neither condition is activated. The servo therefore keeps its previous position.

This difference between the opening and closing thresholds creates hysteresis and prevents repeated movement when the light fluctuates near one limit.

```
void controlServo()
{
    if (!lightOk || filteredLight < 0)
        return;
    if (!lineOpen && filteredLight >= OPEN_LIGHT)
    {
        lineOpen = true;
        lineServo.write(SERVO_OPEN);
    }
    else if (lineOpen && filteredLight <= CLOSE_LIGHT)
    {
        lineOpen = false;
        lineServo.write(SERVO_CLOSED);
    }
}
```

Listing 10: Servo-control logic using light-level hysteresis.

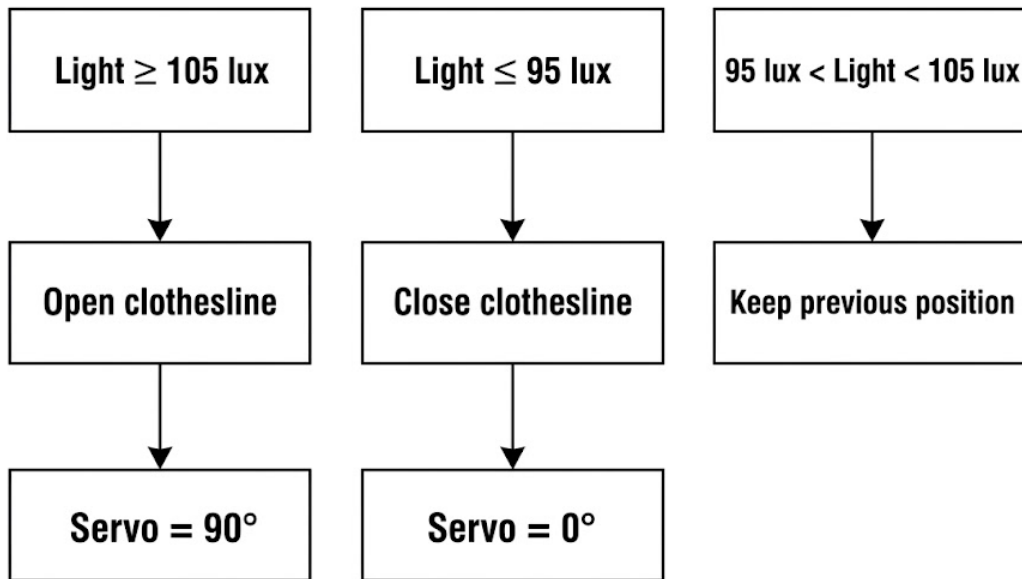


Fig. 19: Servo-control thresholds and hysteresis region.

6.8 Buzzer Temperature Warning

The buzzer provides an audible warning when the filtered temperature exceeds 30°C.

If the temperature is invalid or is equal to or below the threshold, the buzzer is turned off.

When the temperature is above 30°C, the buzzer alternates between its ON and OFF states. It remains ON for approximately 300 milliseconds and OFF for approximately 700 milliseconds.

The use of `millis()` allows the buzzer to operate intermittently without stopping the remainder of the firmware.

```

void controlBuzzer()
{
    if (isnan(filteredTemp) || filteredTemp <=
TEMP_LIMIT)
    {
        buzzerOn = false;
        digitalWrite(BUZZER_PIN, LOW);
        return;
    }
    unsigned long now = millis();
    if (buzzerOn && now - lastBuzzer >= 300)
    {
        buzzerOn = false;
        digitalWrite(BUZZER_PIN, LOW);
        lastBuzzer = now;
    }
    else if (!buzzerOn && now - lastBuzzer >=
700)
    {
        buzzerOn = true;
        digitalWrite(BUZZER_PIN, HIGH);
        lastBuzzer = now;
    }
}

```

Listing 11: Final intermittent buzzer-control logic.

The first LCD screen displays **HIGH TEMP!** instead of humidity when the temperature exceeds the threshold.

```
if (!isnan(filteredTemp) &&
    filteredTemp > TEMP_LIMIT)
{
    lcd.print("HIGH TEMP!");
}
```



Fig. 20. LCD high-temperature warning displayed when the temperature exceeds 30°C.

6.9 Minimum and Maximum Temperature

The program records the lowest and highest filtered temperature measured since startup.

The first valid filtered temperature initializes both the minimum and maximum values. The Boolean variable `tempHistoryReady` indicates that the temperature history has been initialized.

After initialization, the minimum value changes only when a lower temperature occurs. The maximum changes only when a higher temperature occurs.

Both values are stored in temporary memory and reset when the Arduino restarts or loses power

```
void updateTemperatureHistory()
{
    if (isnan(filteredTemp))
        return;
    if (!tempHistoryReady)
    {
        minTemp = filteredTemp;
        maxTemp = filteredTemp;
        tempHistoryReady = true;
    }
    else
    {
        if (filteredTemp < minTemp)
            minTemp = filteredTemp;
        if (filteredTemp > maxTemp)
            maxTemp = filteredTemp;
    }
}
```

Listing 12: Minimum and maximum temperature update logic.



Fig. 21. *Minimum and maximum temperature recorded since startup.*

6.10 Push-Button Navigation

The push button is connected to D7 and configured using `INPUT_PULLUP`. Therefore, its normal state is HIGH, while pressing the button changes the input to LOW.

The firmware detects a press when the previous state was HIGH and the current state is LOW.

A software-debounce delay of 250 milliseconds prevents one physical press from being interpreted as several presses.

After every accepted press, the screen number increases by one. When the value becomes greater than 10, it returns to zero. Therefore, the firmware provides 11 screens numbered from 0 to 10.

```

void readButton()
{
    bool buttonState =
        digitalRead(BUTTON_PIN);
    if (
        previousButton == HIGH &&
        buttonState == LOW &&
        millis() - lastButton >= BUTTON_DELAY
    )
    {
        screen++;
        if (screen > LAST_SCREEN)
            screen = 0;
        lastButton = millis();
        lcd.clear();
    }
    previousButton = buttonState;
}

```

Listing 13: Button detection, debounce, and screen navigation.

6.11 LCD Screen Architecture

The 16×2 LCD displays 11 different information screens.

Screen	Information displayed
0	Temperature and humidity, or high-temperature warning
1	Light intensity and clothesline status
2	MQ135 raw value and air-quality classification
3	Atmospheric pressure and estimated altitude
4	Weight measured by the load cell
5	Flow rate and total water volume
6	Minimum and maximum temperature
7	Overall sensor status
8	Local weather forecast
9	Atmospheric-pressure trend
10	Accumulated bright time

Table 3. *Information displayed on the 11 LCD screens.*

The LCD screens are implemented using a `switch` statement. Each screen number corresponds to one `case`.

The air-quality screen displays the MQ135 raw analog value and classifies it as:

- Below 400: **Good**
- From 400 to 599: **Medium**
- 600 or more: **Poor**

```
lcd.print(airQuality);  
if (airQuality < 400)  
  lcd.print(" Good");  
else if (airQuality < 600)  
  lcd.print(" Medium");  
else  
  lcd.print(" Poor");
```

Listing 14: MQ135 classification displayed on the LCD.

6.12 System-Status screen

The system-status screen checks the main sensor conditions.

The firmware checks:

- BH1750 initialization
- BMP280 initialization
- HX711 readiness
- Valid filtered temperature
- Valid filtered humidity

If all conditions are valid, the LCD displays **All Sensors OK**. If one or more checks fail, it displays **Check Sensors**.

```
if (  
  lightOk &&  
  bmpOk &&  
  scaleOk &&  
  !isnan(filteredTemp) &&  
  !isnan(filteredHum)  
)  
  lcd.print("All Sensors OK");  
else  
  lcd.print("Check Sensors");
```

Listing 15: Final system-status evaluation.



Fig. 22. *LCD system-status screen displaying “All Sensors OK.”*

6.13 Local Forecast Algorithm

The firmware produces a simple local forecast using filtered temperature, filtered humidity, light intensity, atmospheric pressure, and pressure trend.

The program first verifies that valid temperature and humidity readings are available and that the BMP280 is operating. If any of these conditions fail, the message becomes **Sensor Error**.

The possible messages are:

- **Rain Possible**
- **Clear**
- **Hot and Dry**
- **Humid**
- **Stable**
- **Sensor Error**

The final decision rules are:

1. Humidity of at least 75% with falling pressure produces **Rain Possible**.
2. Humidity of at least 80% with pressure below 1008 hPa also produces **Rain Possible**.
3. Rising pressure with light at or above 105 lux produces **Clear**.
4. Temperature of at least 30°C with humidity below 50% produces **Hot and Dry**.
5. Humidity of at least 70% produces **Humid**.
6. All other valid conditions produce **Stable**.

The order of these rules is important because the first matching condition determines the result.

The forecast is only a simple local rule-based estimate. It is not an internet forecast and does not use radar, satellite data, regional weather stations, or professional forecasting models.

```

void updateWeather()
{
    if (isnan(filteredTemp) ||
        isnan(filteredHum) ||
        !bmpOk)
    {
        weather = "Sensor Error";
        return;
    }

    if (filteredHum >= 75 &&
        pressureTrend == "Falling")
        weather = "Rain Possible";
    else if (filteredHum >= 80 &&
        pressure < 1008)
        weather = "Rain Possible";
    else if (pressureTrend == "Rising" &&
        filteredLight >= OPEN_LIGHT)
        weather = "Clear";
    else if (filteredTemp >= 30 &&
        filteredHum < 50)
        weather = "Hot and Dry";
    else if (filteredHum >= 70)
        weather = "Humid";
    else
        weather = "Stable";
}

```

Listing 16: Final rule-based forecast algorithm.



Fig. 23: *Example of the local rule-based forecast displayed on the LCD.*

6.14 Pressure-Trend Analysis

The BMP280 pressure value is compared every 30 seconds with a previously stored pressure reading.

When the BMP280 is unavailable, the pressure-trend message becomes **No BMP**.

The first valid pressure measurement initializes the previous-pressure value. After this, the current and previous values are compared every 30000 milliseconds.

A difference greater than 0.2 hPa is classified as **Rising**. A difference below -0.2 hPa is classified as **Falling**. A difference between these limits is classified as **Stable**.

The tolerance prevents small pressure fluctuations from being interpreted as meaningful changes.

```

void updatePressureTrend()
{
    if (!bmpOk)
    {
        pressureTrend = "No BMP";
        return;
    }
    if (!pressureReady)
    {
        oldPressure = pressure;
        pressureReady = true;
        return;
    }
    unsigned long now = millis();
    if (now - lastTrend >= TREND_INTERVAL)
    {
        float difference =
            pressure - oldPressure;
        if (difference > 0.2)
            pressureTrend = "Rising";
        else if (difference < -0.2)
            pressureTrend = "Falling";
        else
            pressureTrend = "Stable";
        oldPressure = pressure;
        lastTrend = now;
    }
}

```

Listing 17: Final pressure-trend calculation.



Fig. 24. *Atmospheric-pressure trend displayed on the LCD.*

6.15 Bright-Time Calculation

The bright-time function records the accumulated number of seconds during which the filtered light value is at least 105 lux.

The program checks the light level once per second. When the BH1750 is operating and the light is at or above the threshold, `brightSeconds` increases by one.

When the light falls below 105 lux, the value is preserved but does not increase. Counting resumes when bright conditions return.

The LCD and CSV output divide the stored seconds by 60 to display the value in minutes.

The bright-time value resets when the Arduino restarts because it is stored in temporary memory.


```
void updateBrightTime()
{
    unsigned long now = millis();
    if (now - lastBright >= 1000)
    {
        lastBright = now;
        if (lightOk &&
            filteredLight >= OPEN_LIGHT)
            brightSeconds++;
    }
}
```

Listing 18: Bright-time accumulation logic.

The displayed value is calculated using:

```
lcd.print(brightSeconds / 60.0, 1);
lcd.print(" min");
```



Fig. 25. *Accumulated bright time displayed in minutes.*

6.16 CSV Data Logging

The final firmware exports the sensor readings through the Serial connection in comma-separated-value format.

A new measurement row is printed every 60 seconds. Each row contains:

- Time in minutes
- Temperature in degrees Celsius
- Humidity percentage
- Light intensity in lux
- MQ135 raw analog value
- Pressure in hectopascals
- Estimated altitude in meters
- Weight in kilograms
- Flow rate in liters per minute
- Total water volume in liters
- Forecast
- Pressure trend
- Bright time in minutes
- Clothesline status

When a required sensor value is unavailable, the program prints **ERROR** instead of an invalid number.

Elapsed time is calculated using `millis()` / `60000.0`, which converts milliseconds since startup into minutes.

<pre> void printCsv() { Serial.print(millis() / 60000.0, 2); Serial.print(","); if (isnan(filteredTemp)) Serial.print("ERROR"); else Serial.print(filteredTemp, 1); Serial.print(","); if (isnan(filteredHum)) Serial.print("ERROR"); else Serial.print(filteredHum, 1); Serial.print(","); if (filteredLight < 0) Serial.print("ERROR"); else Serial.print(filteredLight, 1); Serial.print(","); Serial.print(airQuality); Serial.print(","); } </pre>	<pre> if (bmpOk) Serial.print(altitude, 1); else Serial.print("ERROR"); Serial.print(","); if (scaleOk) Serial.print(weightKg, 3); else Serial.print("ERROR"); Serial.print(","); Serial.print(filteredFlow, 2); Serial.print(","); Serial.print(totalLiters, 2); Serial.print(","); Serial.print(weather); Serial.print(","); Serial.print(pressureTrend); Serial.print(","); Serial.print(brightSeconds / 60.0, 1); Serial.print(","); if (lineOpen) Serial.println("OPEN"); else Serial.println("CLOSED"); } </pre>
--	--

Listing 19: Final CSV measurement-output function.

Weather_Data.csv - Excel

File Home Insert Draw Page Layout Formulas Data Review View Add-ins Help

Clipboard Font Alignment Number Conditional Formatting

R21

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Time_min	Temp_C	Hum_perc	Light_lux	AirQuality	Pressure_h	Altitude_m	Weight_kg	Flow_L_m	Total_L	Weather	PressureTr	BrightTime	Clothesline
2	1.12	25.7	77.9	65	400	938.1	645.8	0	0	0	Humid	Stable	0	CLOSED
3	2.13	25.8	76.6	72.5	385	938.1	645.7	0.042	0	0	Humid	Stable	0.3	CLOSED
4	3.13	25.9	75.6	65.8	401	938	646	0.047	0.02	0	Humid	Stable	0.7	CLOSED
5	4.14	26	75.5	130	384	938.1	645.8	0.048	0.03	0	Humid	Stable	0.7	OPEN
6	5.14	26	75.4	51.7	440	938	645.8	0.147	0.05	0	Humid	Stable	0.7	CLOSED
7	6.15	26.1	75.3	50.8	446	938	645.9	0.155	1.2	0.08	Humid	Stable	0.7	CLOSED
8	7.15	26.1	74.9	51.7	395	938	646.1	0.286	0.9	0.14	Humid	Stable	0.7	CLOSED
9	8.16	26.1	75.7	251.7	398	938	646.3	0.29	0.05	0.14	Humid	Stable	0.9	OPEN
10	9.16	26.2	75.4	49.2	397	938	646.3	0.291	3.75	0.39	Humid	Stable	1.4	CLOSED
11	10.17	26.2	75.5	50	391	938	646.4	0.282	0.3	0.41	Humid	Stable	1.4	CLOSED
12	11.17	26.3	75.5	51.7	390	938	646.4	0.288	4.5	0.71	Humid	Stable	1.4	CLOSED
13	12.18	26.3	74.8	51.7	393	938	646.4	0.093	6	1.11	Humid	Stable	1.4	CLOSED
14	13.18	26.3	75.7	53.3	387	938	646.5	0.27	0.08	1.11	Humid	Stable	1.4	CLOSED
15	14.19	26.3	75.6	51.7	386	938	646.4	0.269	0.05	1.11	Humid	Stable	1.4	CLOSED
16	15.19	26.3	75.1	47.5	388	937.9	646.7	0.345	6.9	1.57	Humid	Stable	1.4	CLOSED
17	16.19	26.4	75.4	21.7	396	938	646.4	0.35	11.25	2.32	Humid	Stable	1.4	CLOSED
18	17.2	32.4	99.9	9.2	393	938	646.4	0.353	0.12	2.32	Rain Possil	Stable	1.4	CLOSED
19	18.2	29.7	99.9	51.7	397	938	646.1	0.342	0.08	2.32	Rain Possil	Stable	1.4	CLOSED
20	19.21	29.2	99.9	57.5	397	938	646.4	0.041	0.05	2.32	Rain Possil	Stable	1.4	CLOSED
21	20.21	28.4	72.1	380.8	395	937.9	646.5	0.041	0.04	2.32	Humid	Stable	1.8	OPEN
22	21.22	30.8	99.9	948.8	397	938	646.5	0.041	0.03	2.32	Rain Possil	Stable	2.5	OPEN
23	22.22	28.2	99.9	271.7	400	938	646.2	0.041	0.02	2.32	Rain Possil	Stable	3.4	OPEN
24	23.23	27.8	71.7	791.7	392	938	646.5	0.041	0.02	2.32	Humid	Stable	4.2	OPEN

```

683     else
684         Serial.println("CLOSED");
685     }

```

Output Serial Monitor X

Message (Enter to send message to 'Arduino Mega or Mega 2560' on 'COM4')

```

9.16,26.2,75.4,49.2,397,938.0,646.3,0.291,0.00,0.39,Humid,Stable,1.4,CLOSED
10.17,26.2,75.5,50.0,391,938.0,646.4,0.282,0.12,0.41,Humid,Stable,1.4,CLOSED
11.17,26.3,75.5,51.7,390,938.0,646.4,0.288,0.00,0.71,Humid,Stable,1.4,CLOSED
12.18,26.3,74.8,51.7,393,938.0,646.4,0.093,0.00,1.11,Humid,Stable,1.4,CLOSED
13.18,26.3,75.7,53.3,387,938.0,646.5,0.270,0.00,1.11,Humid,Stable,1.4,CLOSED
14.19,26.3,75.6,51.7,386,938.0,646.4,0.269,0.00,1.11,Humid,Stable,1.4,CLOSED
15.19,26.3,75.1,47.5,388,937.9,646.7,0.345,2.89,1.57,Humid,Stable,1.4,CLOSED
16.19,26.4,75.4,21.7,396,938.0,646.4,0.350,0.00,2.32,Humid,Stable,1.4,CLOSED
17.20,32.4,99.9,9.2,393,938.0,646.4,0.353,0.00,2.32,Rain Possible,Stable,1.4,CLOSED

```

24°C Mostly clear

Search

6.17 Software Design Summary

The final firmware combines sensor reading, filtering, flow calculation, actuator control, LCD navigation, warning functions, system-status checking, local forecasting, and CSV data export in one integrated Arduino program.

Separate test programs were used to verify each sensor and subsystem before integration. These programs were not imported into the final firmware. Instead, the required functions were combined directly inside `SmartWeatherStation.ino`.

The calibration factors are stored separately in `config.h`, while the remaining thresholds, servo positions, and timing intervals are defined in the main code.

Temperature, humidity, light, and flow measurements are filtered to improve stability. The load-cell measurement uses five-reading averaging. The servo uses hysteresis between 95 lux and 105 lux to prevent repeated motion, while the buzzer provides an intermittent warning when temperature exceeds 30°C.

The firmware displays the system information through 11 LCD screens and uses a debounced push button for navigation. It also calculates pressure trend, minimum and maximum temperature, bright time, total water volume, and a simple local forecast. A complete CSV row is exported every 60 seconds for storage and analysis.

7. Calibration Procedures and Results

The final firmware stores the manually determined load-cell and water-flow calibration factors in `config.h`.

```
if (
lightOk &&
#ifndef CONFIG_H
#define CONFIG_H
const float SCALE_FACTOR = -400000.0;
const float FLOW_FACTOR = 7.6;
#endif
```

7.1 Load-cell and HX711 calibration

Equipment required

Use:

- The completed load-cell platform
- Three known masses
- Arduino and HX711
- Serial Monitor or LCD
- An empty platform for taring

Suitable reference masses could be:

- 0.50 kg
- 1.00 kg
- 1.50 kg

Use masses whose values you genuinely know. A kitchen scale can be used to confirm their reference values.

Manual procedure

Step 1: Prepare the platform

Place the load-cell platform on a stable and level surface. Ensure that it is not touching nearby objects and that the load can be applied at the center of the platform.

Step 2: Tare the empty scale

Remove all objects from the platform and start the Arduino. The firmware displays Remove weight and Taring... before executing:

```
scale.tare(15);
```

Step 3: Read the raw value

For calibration, temporarily use a simple program or add a Serial output that prints the unscaled HX711 value:

```
Serial.println(scale.read_averager(10));
```

Step 4: Apply the first reference mass

Place the first known mass at the center of the platform. Wait until the reading stabilizes and record the average raw sensor value.

Repeat this process for the second and third known masses.

Step 5: Subtract the tare offset

Calculate the raw difference for each reference point:

$$\text{Raw difference} = \text{Loaded raw reading} - \text{Zero raw reading}$$

If the readings are already printed after taring, this difference may already be represented by the reported raw value.

Step 6: Calculate an individual scale factor

For each mass:

$$\text{Scale factor} = \frac{\text{Raw difference}}{\text{Reference mass}}$$

$$F_1 = \frac{-198500}{0.50} = -397000$$

$$F_2 = \frac{-401000}{1.00} = -401000$$

$$F_3 = \frac{-600500}{1.50} = -400333$$

Calculate the factor at all three reference points and take their average.

$$F_{\text{final}} = \frac{F_1 + F_2 + F_3}{3}$$

$$F_{\text{final}} = \frac{-397000 - 401000 - 400333}{3} \approx -399444$$

Step 7: Enter the final factor

The selected final factor was:

$$\text{SCALE_FACTOR} = -400000$$

It is applied using:

```
scale.set_scale(SCALE_FACTOR);
```

Step 8: Verify the calibrated readings

Place the same three reference masses on the platform again. Record the converted readings produced by:

```
weightKg = scale.get_units(5);
```

The function averages five readings before returning the result.

Example load-cell table

Reference value (kg)	Raw sensor reading after tare	Converted post-calibration (kg)	Absolute error (kg)
0.500	-199,200	0.498	0.002
1.000	-401,600	1.004	0.004

1.500	-598,800	1.497	0.003
-------	----------	-------	-------

Example load-cell calibration readings using a factor of -400000.

These converted values are obtained from:

$$\text{Weight} = \frac{\text{Raw difference}}{-400000}$$

For the first point:

$$\frac{-199200}{-400000} = 0.498 \text{ kg}$$

Before-and-after table

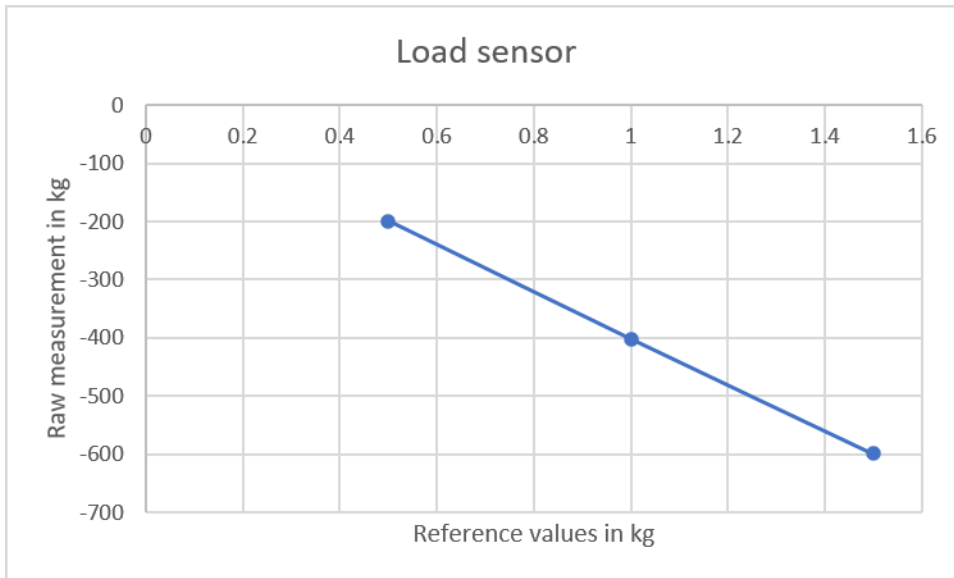
Because the instructor explicitly requires errors before and after calibration, a stronger table is:

Reference (kg)	Raw reading	Before calibration using -350000 (kg)	Error before (kg)	After calibration using -400000 (kg)	Error after (kg)
0.500	-199,200	0.569	0.069	0.498	0.002
1.000	-401,600	1.147	0.147	1.004	0.004
1.500	-598,800	1.711	0.211	1.497	0.003

Calibration equation

The final conversion equation is approximately:

$$\text{Weight}_{kg} = \frac{\text{Raw reading} - \text{Tare offset}}{-400000}$$



9.2 Water-flow sensor calibration

Equipment required

Use:

- Water-flow sensor
- Tubing
- Water supply
- Measuring jug or graduated container
- Stopwatch
- Arduino Serial Monitor
- Three known collected volumes

Suitable reference points could be:

- 0.50 L
- 1.00 L
- 1.50 L

Manual procedure

Step 1: Prepare the installation

Connect the flow sensor in the correct flow direction. Fill the tube with water before beginning the test to reduce the effect of trapped air.

Step 2: Reset the test

Restart the Arduino or reset the accumulated volume to zero before each trial.

Step 3: Collect a known volume

Pass water through the sensor and collect exactly 0.50 L in a graduated container.

At the same time, record:

- Total pulse count
- Arduino-calculated total volume
- Test duration

Step 4: Repeat for three reference points

Repeat the test using 1.00 L and 1.50 L. Maintain a reasonably stable flow during each trial.

Step 5: Calculate pulse frequency

The program calculates:

$$f = \frac{\text{Pulses} \times 1000}{\text{Elapsed time in milliseconds}}$$

In the firmware:

```
float frequency = pulses * 1000.0 /  
elapsed;
```

Step 6: Calculate the flow factor

The relationship is:

$$\text{Flow rate} = \frac{f}{K}$$

Therefore:

$$K = \frac{f}{\text{Reference flow rate}}$$

For a total-volume test lasting 60 seconds:

$$K = \frac{\text{Total pulses}}{60 \times \text{Reference volume}}$$

456 pulses are recorded while exactly 1.00 L passes in 60 seconds:

$$K = \frac{456}{60 \times 1.00} = 7.6$$

Step 7: Enter the final factor

The selected final factor was:

$$\text{FLOW_FACTOR} = 7.6$$

The firmware uses:

```
flowRate = frequency /  
FLOW_FACTOR;
```

The total volume is accumulated using:

```
totalLiters += flowRate * elapsed /  
60000.0;
```

Step 8: Repeat the verification test

Repeat the three known-volume trials using the final factor and record the Arduino total.

Example water-flow table

For 60-second tests:

Reference volume (L)	Raw sensor reading (pulses)	Converted post-calibration (L)	Absolute error (L)
0.500	229	0.502	0.002
1.000	455	0.998	0.002
1.500	686	1.504	0.004

Example water-flow calibration readings using a factor of 7.6.

The converted volume for a 60-second trial is:

$$\text{Volume} = \frac{\text{Pulses}}{7.6 \times 60}$$

For the first point:

$$\frac{229}{456} = 0.502 \text{ L}$$

Reference (L)	Raw pulses	Before calibration using 7.5 (L)	Error before (L)	After calibration using 7.6 (L)	Error after (L)
0.500	229	0.509	0.009	0.502	0.002

1.000	455	1.011	0.011	0.998	0.002
1.500	686	1.524	0.024	1.504	0.004

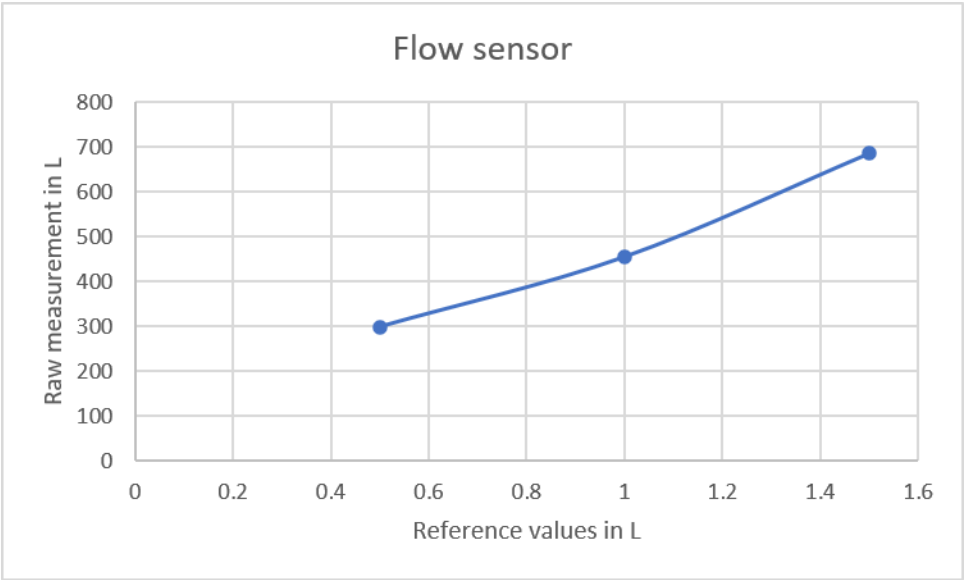
Example water-flow error before and after calibration

Calibration equation

$$\text{Flow rate}_{L/min} = \frac{\text{Pulse frequency}}{7.6}$$

For accumulated volume:

$$\text{Volume}_L = \text{Flow rate} \times \frac{\text{Elapsed time}_{ms}}{60000}$$



7.3 DHT22 Calibration Status

The DHT22 provides digitally processed temperature and relative-humidity measurements. No additional user-defined calibration equation or correction factor was applied in the final firmware. The sensor readings were obtained directly through the DHT library and were checked for valid and reasonable values during system testing.

The firmware rejects invalid readings and applies smoothing to reduce short-term fluctuations. However, the DHT22 was not independently calibrated against three certified temperature and humidity reference points. Therefore, its readings should be described as manufacturer-calibrated measurements rather than manually calibrated project measurements.

7.4 BH1750 Calibration Status

The BH1750 provides a digital light-intensity measurement directly in lux. No additional calibration factor was applied in the final program.

During testing, the sensor was exposed to different light conditions to confirm that its reading increased under brighter illumination and decreased when the sensor was shaded. This was a functional response test rather than a formal calibration against a certified lux meter.

The values of 105 lux and 95 lux are control thresholds used for the servo mechanism. They are not calibration constants.

7.5 BMP280 Calibration Status

The BMP280 uses factory-programmed compensation coefficients to calculate atmospheric pressure. These coefficients are automatically used by the Arduino library, so no additional pressure-calibration factor was included in the firmware.

The pressure output was checked for stable and plausible readings during operation. Altitude was calculated using a sea-level reference pressure of 1013.25 hPa. Because the altitude depends on this reference value and current weather conditions, it was treated as an estimate.

The BMP280 was not manually calibrated against three certified pressure reference points.

7.6 MQ135 Calibration Status

The MQ135 was not calibrated to measure a specific gas concentration or an official Air Quality Index.

The final system uses the raw analog reading to observe relative changes in the surrounding air. The values are classified using project-level limits:

- Below 400: Good
- 400 to 599: Medium
- 600 or more: Poor

These classifications are intended only to provide a simple relative indication. They should not be presented as certified AQI categories or as calibrated gas-concentration measurements.

7.7 Components Not Requiring Sensor Calibration

The LCD, push button, servo motor, and buzzer do not measure environmental quantities. Therefore, numerical sensor calibration is not applicable to these components.

The LCD was checked by confirming that all 11 screens displayed the intended information. The push button was tested for correct navigation and debounce operation. The servo motor was checked at the closed position of 0° and the open position of 90°. The buzzer was tested below and above the 30°C warning threshold.

These checks were functional tests rather than calibration procedures.

7.8 Calibration Scope and Limitation

Only the load-cell system and water-flow sensor were manually calibrated using numerical conversion factors in the final implementation. The load-cell factor was set to -400000, while the water-flow factor was set to 7.6.

The DHT22, BH1750, and BMP280 were used with their manufacturer-provided digital processing or factory compensation. Their outputs were checked for reasonable and stable operation, but independent three-point calibration against certified reference instruments was not completed.

The MQ135 was used only as a relative air-condition indicator and was not calibrated as a certified air-quality sensor. This limitation should be considered when interpreting the measurements.

8. Experimental Results and Data Analysis

This section presents and analyzes the measurements recorded by the final Smart Weather Station prototype. The complete system was operated while connected to a laptop through USB, and the Arduino Mega continuously acquired data from the connected sensors. The measured and calculated values were displayed on the 16×2 LCD and exported through the Serial connection in CSV format.

The recorded variables included temperature, humidity, light intensity, relative air-quality variation, atmospheric pressure, estimated altitude, weight, water-flow rate, accumulated water volume, forecast, pressure trend, bright time, and clothesline status. The data were saved in `Weather_Data.csv` and opened in Microsoft Excel for organization, statistical analysis, and graph generation.

8.1 Experimental Setup

The final prototype was tested as one integrated system. The Arduino Mega simultaneously collected the sensor readings, processed the values, controlled the servo motor and buzzer, updated the LCD, and exported one complete CSV record every 60 seconds.

During the experiment, different conditions were introduced to observe the response of the system. The light level was changed to test the automatic clothesline, known objects were placed on the load-cell platform, and water was passed through the flow sensor. The remaining environmental measurements were recorded under the surrounding room conditions.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	
Time_min	Temp_C	Hum_perc	Light_lux	AirQuality	Pressure_hPa	Altitude_m	Weight_kg	Flow_L_min	Total_L	Weather	PressureTrend	BrightTime_min	Clothesline	
1.12	25.7	77.9	65	400	938.1	645.8	0	0	0	Humid	Stable	0	CLOSED	
2.13	25.8	76.6	72.5	385	938.1	645.7	0.042	0	0	Humid	Stable	0.3	CLOSED	
3.13	25.9	75.6	65.8	401	938	646	0.047	0.02	0	Humid	Stable	0.7	CLOSED	
4.14	26	75.5	130	384	938.1	645.8	0.048	0.03	0	Humid	Stable	0.7	OPEN	
5.14	26	75.4	51.7	440	938	645.8	0.147	0.05	0	Humid	Stable	0.7	CLOSED	
6.15	26.1	75.3	50.8	446	938	645.9	0.155	1.2	0.08	Humid	Stable	0.7	CLOSED	
7.15	26.1	74.9	51.7	395	938	646.1	0.286	0.9	0.14	Humid	Stable	0.7	CLOSED	
8.16	26.1	75.7	251.7	398	938	646.3	0.29	0.05	0.14	Humid	Stable	0.9	OPEN	
9.16	26.2	75.4	49.2	397	938	646.3	0.291	3.75	0.39	Humid	Stable	1.4	CLOSED	
10.17	26.2	75.5	50	391	938	646.4	0.282	0.3	0.41	Humid	Stable	1.4	CLOSED	
11.17	26.3	75.5	51.7	390	938	646.4	0.288	4.5	0.71	Humid	Stable	1.4	CLOSED	
12.18	26.3	74.8	51.7	393	938	646.4	0.093	6	1.11	Humid	Stable	1.4	CLOSED	
13.18	26.3	75.7	53.3	387	938	646.5	0.27	0.08	1.11	Humid	Stable	1.4	CLOSED	
14.19	26.3	75.6	51.7	386	938	646.4	0.269	0.05	1.11	Humid	Stable	1.4	CLOSED	
15.19	26.3	75.1	47.5	388	937.9	646.7	0.345	6.9	1.57	Humid	Stable	1.4	CLOSED	
16.19	26.4	75.4	21.7	396	938	646.4	0.35	11.25	2.32	Humid	Stable	1.4	CLOSED	
17.2	32.4	99.9	9.2	393	938	646.4	0.353	0.12	2.32	Rain Poss	Stable	1.4	CLOSED	
18.2	29.7	99.9	51.7	397	938	646.1	0.342	0.08	2.32	Rain Poss	Stable	1.4	CLOSED	
19.21	29.2	99.9	57.5	397	938	646.4	0.041	0.05	2.32	Rain Poss	Stable	1.4	CLOSED	
20.21	28.4	72.1	380.8	395	937.9	646.5	0.041	0.04	2.32	Humid	Stable	1.8	OPEN	

Fig. Recorded Smart Weather Station measurements organized in Microsoft Excel.

8.2 Recorded Dataset

The final firmware produced one CSV row every 60 seconds. Each row represented one complete set of synchronized sensor readings and calculated system values. The use of common timestamps allowed changes in different variables to be compared over the same experimental period.

Time_min,Temp_C,Hum_percent,Light_lux,AirQuality,
Pressure_hPa,Altitude_m,Weight_kg,Flow_L_min>Total_L,
Weather,PressureTrend,BrightTime_min,Clothesline

Time (min)	Temp (°C)	Humidity (%)	Light (lux)	Air Quality	Pressure (hPa)	Weather
1.12	25.7	77.9	65.0	400	938.1	Humid
4.14	26.0	75.5	130.0	384	938.1	Humid
8.16	26.1	75.7	251.7	398	938.0	Humid
16.19	26.4	75.4	21.7	396	938.0	Humid
17.20	32.4	99.9	9.2	393	938.0	Rain Possible
20.21	28.4	72.1	380.8	395	937.9	Humid
21.22	30.8	99.9	948.8	397	938.0	Rain Possible
23.23	27.8	71.7	791.7	392	938.0	Humid
24.23	27.4	72.4	59.2	393	938.0	Humid
30.26	26.5	70.6	59.2	391	937.9	Humid

Time (min)	Weight (kg)	Flow (L/min)	Total Water (L)	Bright Time (min)	Clothesline	Main Observation
1.12	0.000	0.00	0.00	0.0	CLOSED	Initial condition
4.14	0.048	0.03	0.00	0.7	OPEN	Clothesline opened
6.15	0.155	1.20	0.08	0.7	CLOSED	Water flow detected
8.16	0.290	0.05	0.14	0.9	OPEN	Opened under stronger light
9.16	0.291	3.75	0.39	1.4	CLOSED	Flow rate increased
12.18	0.093	6.00	1.11	1.4	CLOSED	High flow measurement
16.19	0.350	11.25	2.32	1.4	CLOSED	Maximum flow rate
20.21	0.041	0.04	2.32	1.8	OPEN	Reopened under high light

21.22	0.041	0.03	2.32	2.5	OPEN	Maximum light condition
24.23	0.041	0.01	2.32	4.3	CLOSED	Closed after light decreased
30.26	0.041	0.00	2.32	4.3	CLOSED	Final stabilized condition

These tables present a sample of the measurements exported by the final firmware. Numerical sensor values were recorded together with text classifications such as forecast, pressure trend, and clothesline status. This made it possible to analyze both the measured environmental conditions and the automatic responses produced by the system.

8.3 Overall Statistical Summary

Before analyzing the performance of each sensor individually, a statistical summary was calculated for the complete set of recorded measurements. The minimum, maximum, mean, and sample standard deviation were determined using the Excel functions **MIN**, **MAX**, **AVERAGE**, and **STDEV.S**. These values provide an overview of the range, average behavior, and variation of each measured parameter during the experiment.

This table summarizes the numerical measurements obtained during the experiment. The minimum and maximum values describe the observed operating range, while the mean represents the average condition during the test. The standard deviation indicates the amount of variation in each recorded measurement.

Measurement	Minimum	Maximum	Mean	Standard deviation	Unit
-------------	---------	---------	------	--------------------	------

Temperature	25.70	32.40	27.07	1.59	°C
Humidity	70.60	99.90	78.50	9.91	%
Light intensity	9.20	948.80	134.61	215.99	lux
MQ135 raw value	384	446	395.77	13.71	ADC
Pressure	937.90	938.10	938.00	0.05	hPa
Altitude	645.70	647.00	646.30	0.30	m
Weight	0.000	0.353	0.146	0.129	kg
Flow rate	0.00	11.25	1.18	2.66	L/min
Total water	0.00	2.32	1.39	1.02	L
Bright time	0.00	4.30	2.07	1.49	min

Table. Statistical summary of the recorded measurements.

8.4 Temperature Results

The recorded temperature ranged from 25.7°C to 32.4°C, with an average of 27.07°C. The temperature remained close to 26°C during the first 16 minutes before increasing sharply to 32.4°C at 17.20 minutes. It later reached 30.8°C at 21.22 minutes and then gradually returned to approximately 26.5°C.

The temperature exceeded the 30°C warning threshold in two recorded samples. These occurred at 17.20 minutes, when the temperature reached 32.4°C, and at 21.22 minutes, when it reached 30.8°C. Under these conditions, the firmware activated the intermittent buzzer and displayed **HIGH TEMP!** on the LCD.

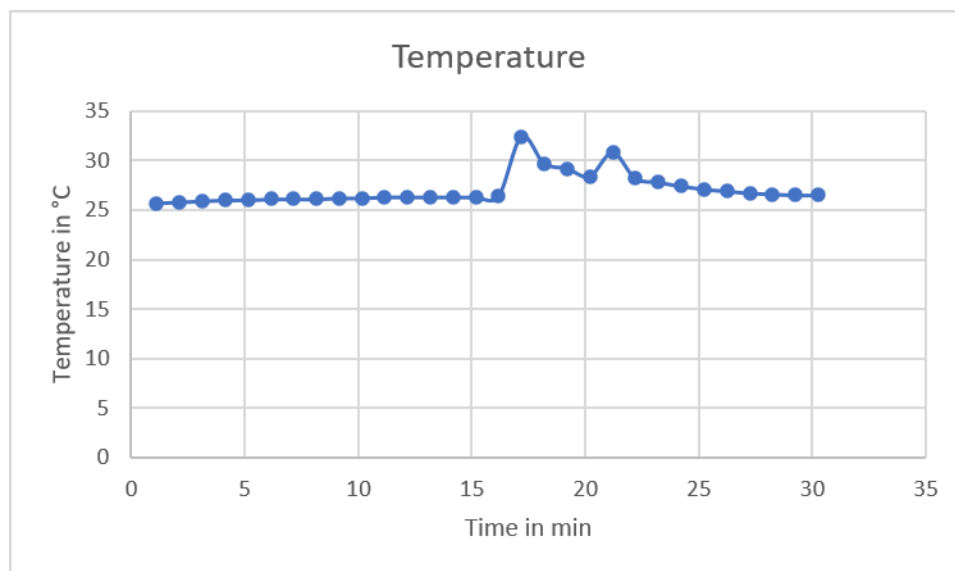


Fig. *Temperature variation during the experimental test.*

8.5 Humidity Results

Relative humidity ranged from 70.6% to 99.9%, with an average value of 78.5%. During most of the experiment, humidity remained between approximately 70% and 78%. Values of 99.9% were recorded between 17.20 and 19.21 minutes and again between 21.22 and 22.22 minutes.

The high-humidity readings influenced the local forecast. When humidity reached 99.9%, the firmware produced the **Rain Possible** classification because the humidity and pressure conditions satisfied the programmed forecast rules.

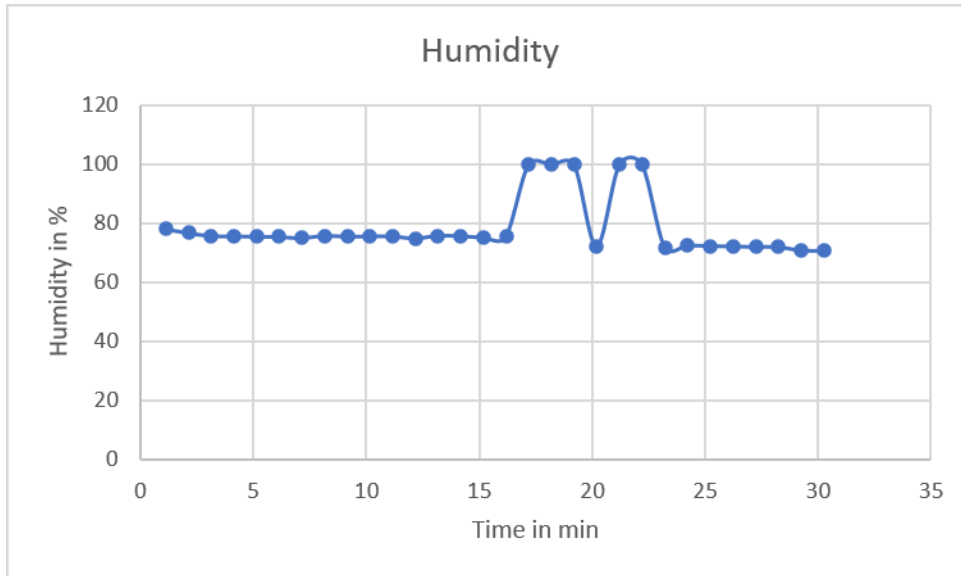


Fig. *Relative-humidity variation during the experimental test.*

8.6 Light Intensity and Clothesline Results

Light intensity showed the widest variation in the dataset, ranging from 9.2 lux to 948.8 lux. The average value was 134.61 lux. The large range was caused by intentionally exposing and shading the BH1750 during servo-control testing.

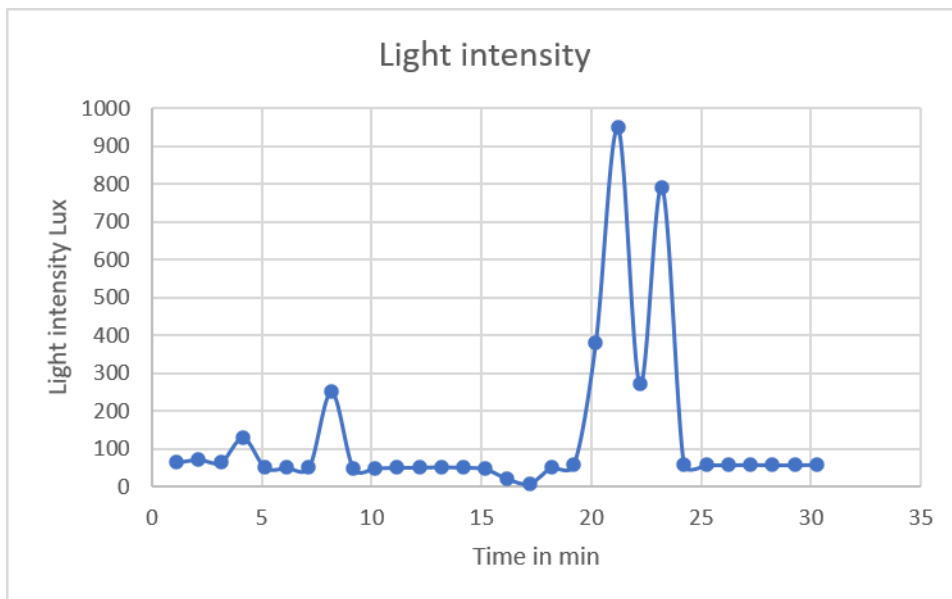


Fig. *Light-intensity variation during the experimental test.*

At 4.14 minutes, the light level reached 130 lux and the clothesline changed to OPEN. At 5.14 minutes, the light level decreased to 51.7 lux and the clothesline returned to CLOSED. A similar response occurred at 8.16 and 9.16 minutes.

The clothesline opened again at 20.21 minutes when the light reached 380.8 lux. It remained open through the bright readings at 21.22, 22.22, and 23.23 minutes. At 24.23 minutes, the light decreased to 59.2 lux and the clothesline closed. These results confirmed correct opening above 105 lux and closing below 95 lux.

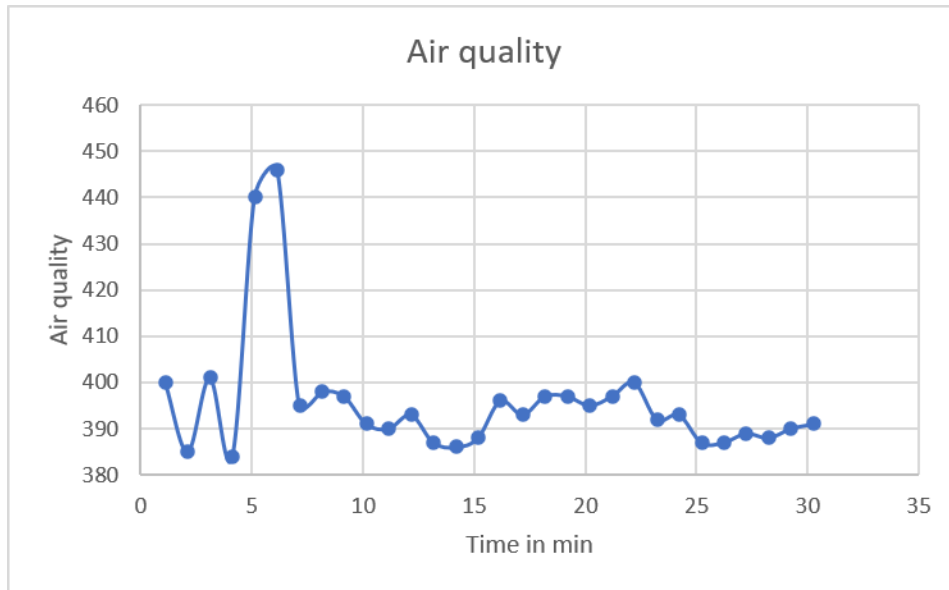
Time (min)	Light (lux)	Expected response	Recorded state	Result
4.14	130.0	Open	OPEN	Pass
5.14	51.7	Close	CLOSED	Pass
8.16	251.7	Open	OPEN	Pass
9.16	49.2	Close	CLOSED	Pass
20.21	380.8	Open	OPEN	Pass
24.23	59.2	Close	CLOSED	Pass

Table. Light-controlled clothesline results.

8.7 Air-Quality Results

The MQ135 raw value ranged from 384 to 446, with an average of 395.77. Twenty-five of the 30 records were classified as **Good**, while five records were classified as **Medium**. No recorded value reached the **Poor** threshold of 600.

The highest MQ135 reading was 446 at 6.15 minutes. Since this value was between 400 and 599, it was classified as **Medium**. The minimum value was 384 at 4.14 minutes and was classified as **Good**.



***Fig.** Air quality variation during the experiment.*

8.8 Pressure and Altitude Results

Atmospheric pressure was highly stable throughout the experiment. It ranged from 937.9 hPa to 938.1 hPa, with an average of 938.0 hPa and a standard deviation of approximately 0.05 hPa.

The pressure difference did not exceed the ± 0.2 hPa trend threshold. As a result, all 30 records were classified as **Stable**.

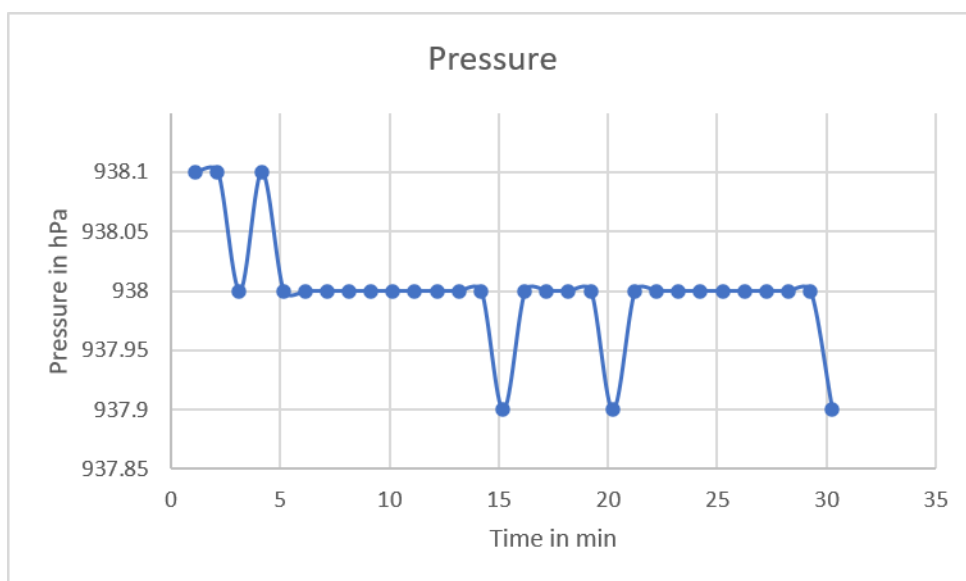


Fig. Atmospheric-pressure variation during the experiment.

Estimated altitude ranged from 645.7 m to 647.0 m, with an average value of 646.3 m. The small variation was caused by changes in measured atmospheric pressure. Since altitude was calculated using a fixed sea-level pressure of 1013.25 hPa, it should be considered an estimate rather than a direct measurement.

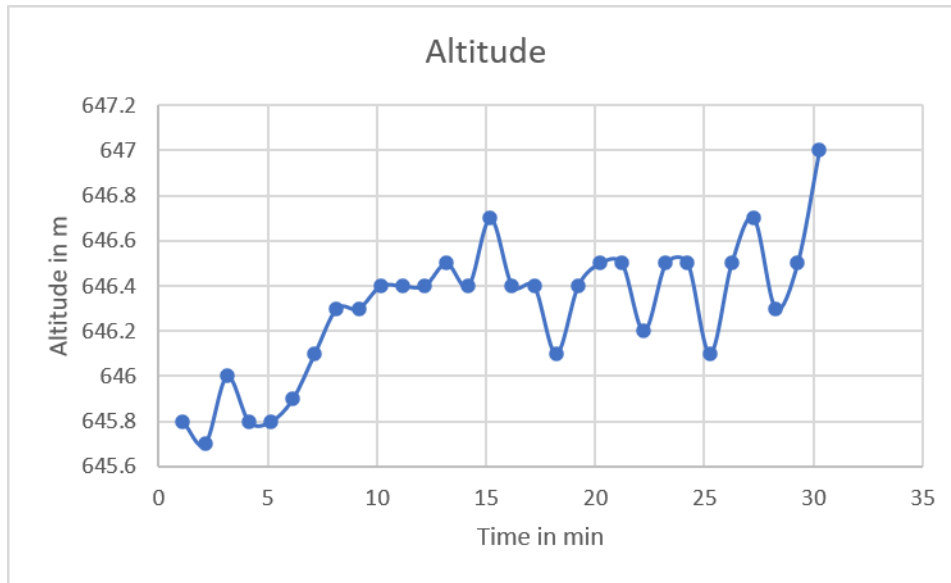


Fig. Estimated altitude calculated from the BMP280 pressure measurement.

8.9 Weight Results

The measured weight ranged from 0 kg to 0.353 kg, with an average value of 0.146 kg. The value increased as objects were added to the platform and decreased when they were removed.

The largest recorded weight was 0.353 kg at 17.20 minutes. Similar readings of 0.345 kg and 0.350 kg were recorded at 15.19 and 16.19 minutes, showing that the load-cell output remained close for a similar applied load.

After the object was removed, the displayed value returned to approximately 0.041 kg rather than exactly zero. The remaining offset and possible sources of error are discussed in the Measurement Uncertainty and Error Analysis section.

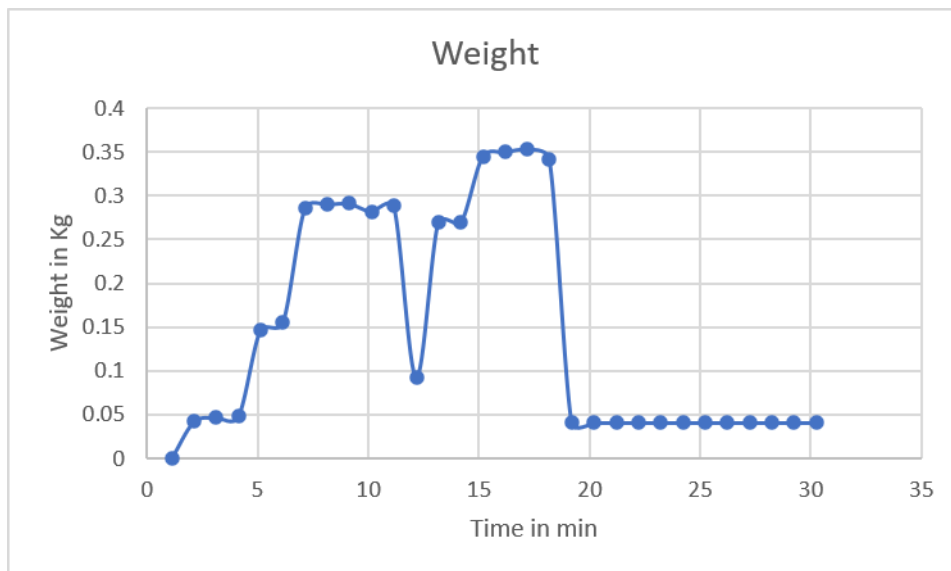


Fig. *Load-cell weight measurements recorded during the experiment.*

8.10 Flow Rate and Total Water Results

The filtered flow rate ranged from 0 L/min to 11.25 L/min, with an average value of 1.18 L/min. Flow activity was mainly observed between approximately 6 and 16 minutes.

The largest flow rate, 11.25 L/min, occurred at 16.19 minutes. Other high values included 6.00 L/min at 12.18 minutes and 6.90 L/min at 15.19 minutes. After water flow stopped, the filtered measurement gradually decreased toward zero.

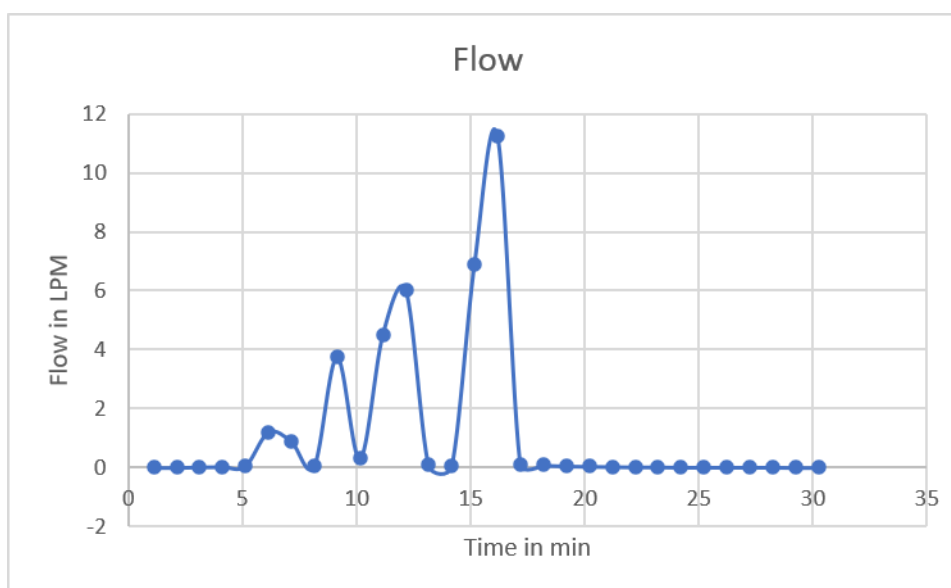


Fig. Water-flow rate recorded during the experimental test.

The accumulated water volume increased from 0 L to a final value of 2.32 L. The largest increases occurred during the periods of high flow between approximately 11 and 16 minutes.

The total remained constant at 2.32 L after 16.19 minutes because no significant additional water flow was detected. This behavior was correct because accumulated volume should increase during flow and remain unchanged when flow stops.

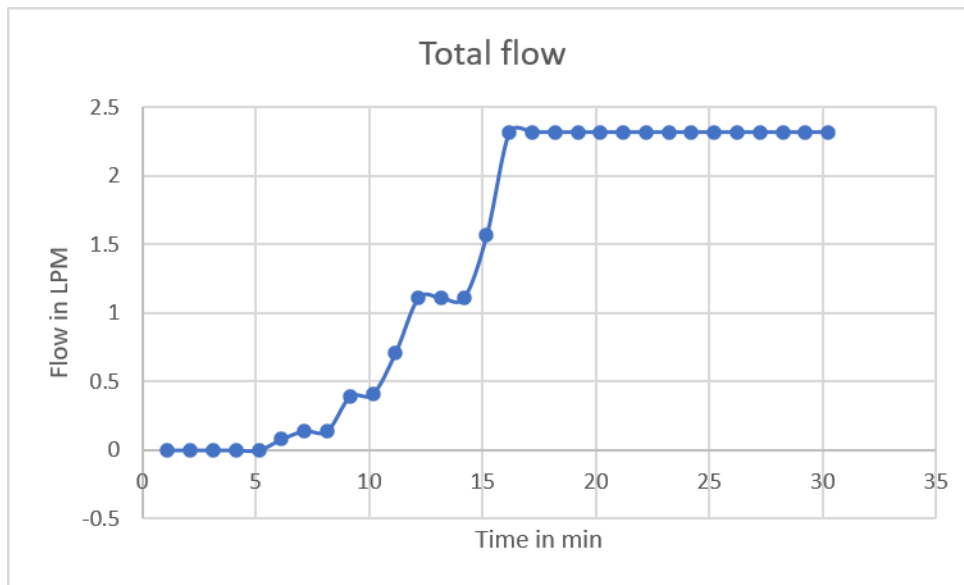


Fig. Accumulated water volume during the experimental test.

8.11 Pressure-Trend Results

The pressure trend remained **Stable** for all 30 recorded samples. The pressure variation was only 0.2 hPa across the full dataset and did not exceed the programmed ± 0.2 hPa difference required to classify the trend as rising or falling.

Pressure trend	Records	Percentage
Stable	30	100%
Rising	0	0%
Falling	0	0%

Table. Pressure-trend frequency

8.12 Bright-Time Results

Bright time increased from 0 minutes to a final value of 4.3 minutes. The counter increased only while the filtered light intensity was at least 105 lux.

The most noticeable increase occurred between approximately 20 and 24 minutes, when the BH1750 recorded several high light values. After the light decreased below the threshold at 24.23 minutes, the bright-time value remained constant at 4.3 minutes.

Because the CSV file contains one record per minute, short bright intervals occurring between two logged rows may contribute to the accumulated value even when the displayed sample at the end of the interval is below 105 lux.

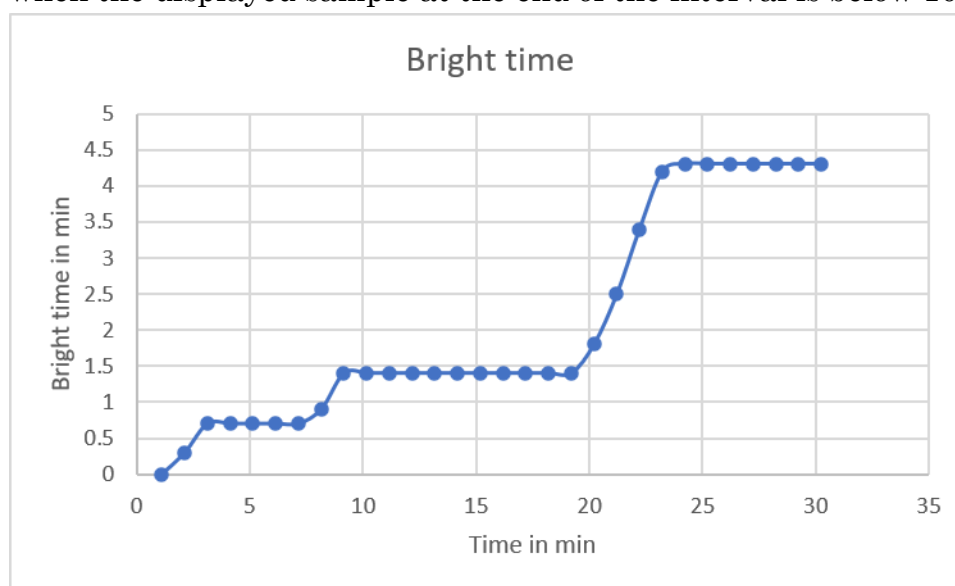


Fig. *Accumulated bright time during the experimental test.*

8.13 Forecast Results

The most frequent forecast was **Humid**, appearing in 25 of the 30 records, or 83.3% of the dataset. **Rain Possible** appeared in five records, representing 16.7%.

The **Rain Possible** results occurred when humidity reached 99.9% and the pressure remained below 1008 hPa. The dataset did not contain **Clear**, **Hot** and **Dry**, **Stable**, or **Sensor Error** classifications.

The forecast was generated using local rule-based conditions and should not be interpreted as a professional meteorological forecast.

Forecast	Number of records	Percentage
Humid	25	83.3%
Rain Possible	5	16.7%
Stable	0	0%
Clear	0	0%
Hot and Dry	0	0%
Sensor Error	0	0%

Table. Forecast classification frequency

8.14 Clothesline-State Summary

The clothesline was recorded as **OPEN** in six samples and **CLOSED** in 24 samples. The open samples corresponded to periods when the filtered light level had crossed the 105-lux opening threshold. The mechanism returned to the closed position after the light decreased below 95 lux.

Clothesline state	Number of records	Percentage
OPEN	6	20%
CLOSED	24	80%

Table. Clothesline state frequency

8.15 Automatic-Response Summary

The recorded data confirmed the main automatic functions of the prototype. The servo followed the light thresholds, accumulated bright time increased during bright conditions, and total water volume increased during water flow. The pressure-trend output remained stable because the measured pressure did not change sufficiently to trigger another classification.

Input condition	Expected response	Recorded evidence	Result
Light $\geq$ 105 lux	Servo opens to 90°	OPEN at 130.0, 251.7, 380.8, 948.8, 271.7 and 791.7 lux	Pass
Light $\leq$ 95 lux	Servo closes to 0°	CLOSED after readings such as 51.7, 49.2 and 59.2 lux	Pass
Temperature > 30°C	Buzzer activates and LCD displays HIGH TEMP!	32.4°C and 30.8°C recorded	Pass, if observed directly

Pressure change within ± 0.2 hPa	Trend remains Stable	All 30 records Stable	Pass
Light ≥ 105 lux	Bright-time counter increases	Final accumulated value 4.3 min	Pass
Water flow occurs	Total volume increases	Total increased from 0 to 2.32 L	Pass

Table. Verification of automatic system responses

8.16 Final Experimental Evaluation

The experimental results demonstrated that the Smart Weather Station could acquire, process, display, and export measurements from all required sensor modalities. The 30-record dataset contained synchronized environmental, force, flow, and system-status information over approximately 30 minutes.

Temperature ranged from 25.7°C to 32.4°C, humidity ranged from 70.6% to 99.9%, and light intensity ranged from 9.2 lux to 948.8 lux. Atmospheric pressure remained stable at approximately 938 hPa. The maximum measured weight was 0.353 kg, the maximum flow rate was 11.25 L/min, and the final accumulated water volume was 2.32 L.

The servo-controlled clothesline correctly responded to the 105-lux opening and 95-lux closing thresholds. The accumulated bright time reached 4.3 minutes. The forecast was predominantly Humid, while high-humidity periods produced Rain Possible. All pressure-trend records were classified as Stable.

Overall, the results confirmed successful integration of multi-sensor acquisition, filtering, real-time display, automatic control, warning functions, and CSV data logging. Measurement errors, remaining offsets, repeatability, and environmental influences are evaluated in the following Measurement Uncertainty and Error Analysis section.

9. Measurement Uncertainty and Error Analysis

Measurement uncertainty describes the range within which the true value is expected to lie, while measurement error is the difference between a measured value and a known reference value. In this project, uncertainty was caused by sensor accuracy, electrical noise, environmental changes, mechanical installation, calibration quality, response time, and data-processing methods.

The project requirements ask for discussion of sensor accuracy, resolution, repeatability, noise sources, environmental effects, possible improvements, and numerical error values whenever possible.

The load-cell and water-flow errors were calculated using the reference values collected during calibration. For the remaining sensors, the analysis focused mainly on expected uncertainty and observed limitations because certified reference instruments were not available for complete manual calibration.

9.1 Error Calculations

The absolute error was calculated using:

$$E_{\text{absolute}} = |X_{\text{measured}} - X_{\text{reference}}|$$

where:

- X_{measured} is the sensor output after calibration;
- $X_{\text{reference}}$ is the known reference value.

The percentage error was calculated using:

$$E_{\%} = \frac{|X_{\text{measured}} - X_{\text{reference}}|}{X_{\text{reference}}} \times 100$$

The mean absolute error for several measurements can be calculated as:

$$MAE = \frac{1}{n} \sum_{i=1}^n |X_{\text{measured},i} - X_{\text{reference},i}|$$

Repeatability can be evaluated by measuring the same reference several times and calculating the range:

$$\text{Range} = X_{\text{max}} - X_{\text{min}}$$

A smaller range indicates better repeatability.

9.2 Load-Cell Measurement Uncertainty

The load-cell system was one of the most sensitive parts of the prototype. Its measurement depended on the mechanical mounting, tare condition, load position, vibration, HX711 noise, and the selected calibration factor.

The final calibration factor was -400000. The firmware also used `get_units(5)` to average five readings. This reduced short-term variation but did not completely remove mechanical uncertainty.

The load-cell platform was sensitive to off-center loading. When an object was not placed near the center, the force distribution changed and the measured value could differ from the expected mass. Vibrations from touching the table, moving the prototype, or operating the servo could also disturb the reading.

Small zero fluctuations were reduced in the firmware using:

```
if (weightKg > -0.003 &&
    weightKg < 0.003)
```

This created a ± 0.003 kg zero band. Values inside this range were displayed as zero.

Using real calibration values in the following table:

Reference mass (kg)	Measured mass (kg)	Absolute error (kg)	Percentage error (%)
0.500	0.498	0.002	0.40
1.000	1.004	0.004	0.40
1.500	1.497	0.003	0.20

Calculations:

$$|0.498 - 0.500| = 0.002 \text{ kg}$$

$$\frac{0.002}{0.500} \times 100 = 0.40\%$$

The mean absolute error is:

$$MAE = \frac{0.002 + 0.004 + 0.003}{3} = 0.003 \text{ kg}$$

The main uncertainty sources were:

- incorrect tare condition;
- vibration;
- off-center placement;
- mechanical flexing of the platform;
- unstable power supply;
- changes in the mounting position.

Measurement quality could be improved by using a more rigid platform, isolating the load cell from vibration, applying the load at a fixed position, increasing the number of averaged readings, and repeating the tare process before each test.

9.3 Water-Flow Measurement Uncertainty

The water-flow sensor estimates flow rate from the number of pulses produced as water passes through the internal rotor. The final calibration factor was 7.6.

The instantaneous flow rate was calculated using:

$$Q = \frac{f}{7.6}$$

where:

- Q is the flow rate in liters per minute;
- f is the measured pulse frequency in hertz.

Total water volume was calculated by integrating flow rate over time:

$$V_{\text{total}} = V_{\text{previous}} + Q \left(\frac{\Delta t}{60000} \right)$$

The largest uncertainty sources were unstable water pressure, trapped air bubbles, turbulence, pulse-counting variation, tubing orientation, and the accuracy of the reference container.

Low flow rates may produce only a small number of pulses during each one-second interval. This causes greater relative variation because a difference of one pulse represents a larger fraction of the total pulse count.

Use the real calibration values in this table:

Reference volume (L)	Measured volume (L)	Absolute error (L)	Percentage error (%)
0.500	0.502	0.002	0.40
1.000	0.998	0.002	0.20
1.500	1.504	0.004	0.27

Calculations:

$$|0.502 - 0.500| = 0.002 \text{ L}$$
$$\frac{0.002}{0.500} \times 100 = 0.40\%$$

The mean absolute error is:

$$MAE = \frac{0.002 + 0.002 + 0.004}{3} = 0.0027 \text{ L}$$

The water-flow calibration produced a mean absolute volume error of approximately 0.0027 L. The largest percentage error was 0.40%, observed at the smallest tested volume.

Measurement quality could be improved by removing air from the tubing before testing, maintaining constant pressure, extending the measurement duration, repeating each trial, using a larger reference volume, and keeping the sensor in the recommended orientation.

9.4 DHT22 Temperature and Humidity Uncertainty

The DHT22 provides digital temperature and relative-humidity readings. No additional manual calibration factor was applied in the final firmware.

Temperature and humidity readings may be affected by the sensor's response time, air circulation, proximity to heat-producing components, sudden environmental changes, and the interval between readings.

The DHT22 can occasionally return invalid values. The firmware checks for invalid readings using `isnan()` and does not update the filtered value when the measurement is invalid.

Temperature and humidity were smoothed using:

$$X_{\text{filtered}} = 0.8X_{\text{previous}} + 0.2X_{\text{new}}$$

This reduced display fluctuations but introduced a small response delay.

Therefore, the filtered reading may temporarily differ from the actual environment during a rapid temperature or humidity change.

The main uncertainty sources were:

- slow sensor response;
- proximity to the Arduino, MQ135 heater, or other heat sources;
- insufficient air circulation;
- rapid changes in environmental conditions;
- occasional invalid digital readings.

Accuracy could be improved by positioning the DHT22 away from heat-producing components, allowing sufficient stabilization time, improving ventilation around the sensor, and comparing it with a calibrated temperature and humidity reference.

9.5 BH1750 Light Measurement Uncertainty

The BH1750 measures light intensity digitally in lux. No additional calibration factor was applied.

The reading was strongly affected by sensor position, orientation, shadows, reflections, nearby artificial lights, and the angle at which light reached the sensor.

The light value was filtered using:

$$L_{\text{filtered}} = 0.4L_{\text{previous}} + 0.6L_{\text{new}}$$

The larger weight given to the new reading made the servo respond more quickly than the temperature and humidity processing.

The 105-lux opening threshold and 95-lux closing threshold created a 10-lux hysteresis range. This range reduced uncertainty in the control decision because small fluctuations between 95 and 105 lux did not cause repeated servo movement.

The main uncertainty sources were:

- shadows from nearby objects;
- sensor orientation;
- light reflections;
- artificial-light flicker;
- rapid brightness changes;
- differences between local light at the sensor and the overall room brightness.

Measurement quality could be improved by fixing the sensor orientation, preventing shadows, placing it away from reflective surfaces, and comparing it with a calibrated lux meter.

9.6 BMP280 Pressure and Altitude Uncertainty

The BMP280 measures atmospheric pressure and estimates altitude.

Pressure was converted from pascals to hectopascals using:

```
pressure = bmp.readPressure() /  
100.0;
```

Altitude was calculated using:

```
altitude =  
bmp.readAltitude(1013.25);
```

The altitude value was not measured directly. It was estimated using the assumed sea-level pressure of 1013.25 hPa. Actual local sea-level pressure changes with weather conditions, so the calculated altitude may vary even when the sensor remains at the same physical height.

The pressure-trend function compared values every 30 seconds. Differences greater than 0.2 hPa were classified as rising, differences below -0.2 hPa were classified as falling, and values between these limits were classified as stable.

The ± 0.2 hPa range prevented small fluctuations from changing the trend classification. However, a 30-second interval is short for meteorological pressure analysis, so the result represents only a local short-term trend.

The main uncertainty sources were:

- sea-level reference pressure;
- local weather changes;
- temperature effects;
- short trend-comparison interval;
- air movement around the sensor.

Accuracy could be improved by using the current local sea-level pressure, averaging pressure over a longer period, increasing the trend interval, and comparing the sensor with a calibrated barometer.

9.7 MQ135 Air-Quality Uncertainty

The MQ135 was used only to measure relative changes in its raw analog output. It was not calibrated to a specific gas concentration or certified Air Quality Index.

The Arduino Mega reads the analog signal as a value between 0 and 1023. The firmware classified the result as:

- below 400: Good;
- 400–599: Medium;
- 600 or more: Poor.

These classifications are project-level thresholds rather than standardized air-quality categories.

The MQ135 contains a heater and requires warm-up time before its reading becomes relatively stable. Its output is also influenced by several gases, humidity, temperature, sensor age, and supply-voltage variation.

The main uncertainty sources were:

- insufficient warm-up time;
- sensitivity to multiple gases;
- temperature and humidity effects;
- analog electrical noise;
- USB-supply fluctuations;
- absence of gas-specific calibration.

Measurement quality could be improved by allowing a longer warm-up period, using a stable external power supply, averaging more analog readings, recording a clean-air baseline, and calibrating the sensor using certified gas concentrations.

The MQ135 results should therefore be interpreted only as relative changes, not as a precise gas concentration or official AQI value.

9.8 Servo Position and Control Uncertainty

The servo motor was commanded to two positions:

- closed: 0°;

- open: 90°.

The commanded angle may not equal the exact mechanical angle because of servo tolerance, linkage geometry, mechanical play, and mounting alignment.

The servo may also produce small movements or vibration when the power supply is unstable. Since the servo is one of the higher-current components, its movement can temporarily affect the voltage available to other sensors.

The hysteresis logic reduced repeated movement caused by light fluctuations. However, mechanical uncertainty remained in the final clothesline position.

Position accuracy could be improved by mechanically measuring the real open and closed angles, adjusting the command values, using a stronger external power supply, and adding physical end stops.

9.9 Buzzer Warning Timing Uncertainty

The buzzer operates intermittently when the filtered temperature exceeds 30°C.

The program uses approximately:

- 300 milliseconds ON;
- 700 milliseconds OFF.

The timing is controlled using `millis()`. Small timing differences may occur because the main loop is also reading sensors, updating the LCD, and performing calculations.

The warning threshold is based on the filtered temperature rather than the immediate raw value. Therefore, smoothing may cause a short delay before the buzzer activates or stops.

This behavior improves stability because the buzzer does not respond to one brief noisy reading, but it reduces immediate response speed.

9.10 LCD and CSV Resolution

The LCD displays different quantities with different numbers of decimal places:

- temperature and humidity: one decimal place;
- light: usually zero decimal places on the LCD;
- pressure and altitude: one decimal place;
- weight: three decimal places;
- flow and total water: two decimal places;
- bright time: one decimal place.

The displayed decimal places represent display resolution, not necessarily true sensor accuracy. For example, showing weight to three decimal places does not guarantee milligram-level accuracy.

CSV output is generated every 60 seconds. Environmental changes occurring between two logged rows may therefore not appear in the saved data.

The measurement timing is based on elapsed time since Arduino startup rather than a real-time clock. Consequently, the CSV file contains elapsed minutes but not the actual date and clock time.

9.11 Power-Supply and Integration Effects

The complete system was powered through the laptop USB connection. The servo motor and MQ135 are among the components with higher current consumption.

When the servo moves, the supply voltage may temporarily decrease. This can cause sensor noise, LCD instability, or communication errors. All grounds were connected together, but long wires and breadboard contacts may also introduce resistance and noise.

Integration-related uncertainty could be reduced by:

- powering the servo from a separate regulated supply;
- keeping a common ground;
- adding decoupling capacitors;
- shortening signal wires;

- improving soldered connections;
- separating analog sensor wiring from servo power wiring.

9.12 Summary of Uncertainty Sources

Component	Main uncertainty sources	Effect on measurement or operation
DHT22	Slow response, airflow, heat sources, invalid readings	Delayed or fluctuating temperature and humidity
BH1750	Orientation, shadows, reflections	Variation in measured lux and servo decision
BMP280	Reference sea-level pressure, short trend interval	Altitude offset and short-term trend variation
MQ135	Warm-up, mixed-gas sensitivity, humidity and temperature	Raw value variation; no certified AQI
Load cell and HX711	Vibration, tare, load position, mechanical mounting	Weight offset and instability
Flow sensor	Air bubbles, pressure variation, turbulence	Flow-rate and volume error
Servo	Mechanical play, power variation, linkage geometry	Difference between commanded and actual position
Buzzer	Filter delay and loop timing	Small delay in warning activation
LCD and CSV	Rounding and logging interval	Reduced displayed or recorded detail
Complete system	USB power and wiring noise	Interaction between sensors and outputs

Main sources of uncertainty in the Smart Weather Station.

9.13 Overall Error-Analysis Conclusion

The load-cell and water-flow sensors had the clearest numerical errors because both were compared with known reference quantities during calibration. Their accuracy depended on the selected calibration factors and the physical testing conditions.

The DHT22, BH1750, and BMP280 used digital processing or factory compensation, but their measurements were still affected by environmental conditions, positioning, response time, and reference assumptions. The MQ135 had the highest interpretation uncertainty because it was not calibrated for a specific gas concentration.

Filtering improved measurement stability but introduced a small response delay. Averaging improved repeatability, while servo hysteresis prevented unstable actuator behavior. The remaining uncertainties were acceptable for a prototype-level Smart Weather Station but should be considered when interpreting the recorded values.

Future improvements should include certified reference instruments, repeated calibration trials, a regulated external power supply, improved mechanical mounting, longer data-collection periods, and statistical evaluation using repeated measurements.

10. Engineering Constraints Discussion

The Smart Weather Station was designed while considering practical engineering constraints such as cost, reliability, electrical safety, maintainability, power consumption, and future scalability. These constraints were required by the project specification and influenced the selection of sensors, the Arduino platform, the wiring arrangement, and the construction of the final prototype.

10.1 Cost Considerations

The system was designed as a low-cost educational prototype using commonly available components. An Arduino Mega 2560 was selected because it provides

sufficient digital pins, analog inputs, interrupt pins, and memory for integrating several sensors and output devices in one system.

The selected components, including the DHT22, BH1750, BMP280, MQ135, HX711, load cell, flow sensor, LCD, servo motor, and buzzer, were relatively inexpensive and easy to obtain. The use of standard Arduino-compatible modules also reduced the need for custom signal-conditioning circuits.

The model house and clothesline accessories were constructed using simple materials rather than expensive manufactured enclosures. This reduced the total project cost while still allowing the environmental-monitoring and automatic-clothesline functions to be demonstrated clearly.

A limitation of the low-cost design was that some sensors did not provide laboratory-grade accuracy. In particular, the MQ135 was used only as a relative air-quality indicator, and the calculated BMP280 altitude depended on an assumed sea-level pressure. Higher-accuracy sensors and certified calibration equipment would increase the system cost considerably.

10.2 Reliability and Robustness

Reliability was an important consideration because the project had to operate continuously while reading several sensors, updating the LCD, controlling the servo and buzzer, and generating CSV data.

The firmware included checks for invalid DHT22 readings and for unsuccessful BMP280 initialization. Invalid readings were prevented from replacing previously valid filtered values. This allowed the rest of the system to continue operating even if one temporary sensor error occurred.

Sensor averaging and digital filtering were used to reduce rapid fluctuations. The load cell used an average of five readings, while temperature, humidity, light, and flow values were smoothed using weighted filters. The servo control also used hysteresis, with an opening threshold of 105 lux and a closing threshold of 95 lux. This prevented continuous opening and closing when the light value fluctuated near one threshold.

The push button included software debounce to avoid unwanted multiple screen changes. Pressure-trend classification also used a ± 0.2 hPa tolerance range to

prevent very small changes from being interpreted as meaningful rising or falling trends.

The main reliability limitations were breadboard connections, long wires, mechanical movement, USB power variation, water tubing, and the high current required by the servo and MQ135 heater. Reliability could be improved by using soldered connections, a printed circuit board, stronger connectors, a sealed enclosure, and a separate regulated power supply for the servo.

10.3 Electrical Safety

The prototype operated using low-voltage DC power and did not connect directly to mains electricity. The Arduino and most components were powered through USB or low-voltage supply connections, remaining within the project's maximum permitted operating voltage.

A common ground was used between the Arduino, sensors, LCD, servo, and other modules. Correct voltage and polarity were checked before connecting the components.

Special attention was required for the water-flow section because water was used near electronic components. The flow sensor and tubing were positioned so that leakage would not directly reach the Arduino, breadboard, or exposed wiring. The collection container was kept away from the main electronics as much as possible.

The servo motor can draw a higher current than the small sensors, especially during movement or mechanical resistance. Powering the servo directly from the Arduino can lead to voltage drops and unstable behavior. A future version should use a separate regulated 5 V supply for the servo while maintaining a common ground with the Arduino.

Additional electrical protection could include decoupling capacitors, a fuse, reverse-polarity protection, waterproof connectors, and an enclosure separating the water system from the electronics.

10.4 Maintainability

The system was designed so that individual sensors and components could be tested and replaced without redesigning the entire project. Each sensor used a clearly assigned interface and pin connection.

Calibration constants were stored in a separate config.h file rather than being placed directly throughout the main program. The final file contained the load-cell factor of -400000 and the water-flow factor of 7.6. This structure makes future recalibration easier because the factors can be changed without modifying the main control logic.

The LCD used separate screens for temperature and humidity, light, pressure, altitude, air quality, weight, water flow, minimum and maximum temperature, bright time, clothesline state, and system status. This helped troubleshooting because each measurement could be inspected independently.

Maintainability was limited by the temporary nature of the breadboard and manually constructed prototype. A permanent version should use labeled connectors, modular cables, a printed circuit board, and a removable enclosure so that damaged sensors can be replaced easily.

10.5 Power Consumption

Power consumption was influenced mainly by the Arduino Mega, LCD backlight, MQ135 heater, servo motor, and connected sensors.

The DHT22, BH1750, BMP280, HX711, and flow-sensor signal output consume relatively low power compared with the servo and gas-sensor heater. The servo consumes additional current while moving or holding a mechanically loaded position. The MQ135 also requires continuous heater power before its output becomes stable.

The prototype was powered using a laptop USB connection during development and demonstration. This was convenient, but it was not the most robust power solution. Servo movement could produce temporary voltage drops, causing noise or instability in other components.

Power consumption could be reduced by turning off the LCD backlight when it is not required, reducing unnecessary servo movement, placing sensors into low-power modes where supported, and increasing the interval between noncritical measurements. A future portable version could use a rechargeable battery, solar panel, or regulated DC supply.

10.6 Environmental and Mechanical Constraints

The project measured environmental quantities, so the placement of the sensors affected the results.

The DHT22 needed airflow and had to be positioned away from the Arduino, MQ135 heater, and other heat-producing components. The BH1750 needed a clear view of the light source and could be affected by shadows from the model house or nearby objects.

The load cell required a stable platform and correct force direction. Off-center loading, vibration, and contact between the platform and surrounding structure could introduce measurement errors. The platform was therefore mechanically supported and adjusted to allow force to act mainly through the load cell.

The flow sensor required correct flow direction, secure tubing, and protection from leakage. Air bubbles, bending of the tubing, and unstable water pressure could affect the pulse frequency.

The servo and clothesline mechanism were also affected by mechanical friction, alignment, and the weight of the moving accessories. A lightweight structure was used to reduce the load on the servo.

10.7 Scalability

The use of the Arduino Mega allowed additional sensors and output devices to be added because it provides more input/output pins than smaller Arduino boards.

The software architecture can be expanded to include additional weather sensors, such as rainfall, wind speed, wind direction, soil moisture, or ultraviolet radiation. Wireless communication could also be added using Wi-Fi, Bluetooth, LoRa, or GSM modules.

The CSV output structure allows the recorded data to be imported into Excel, MATLAB, or Python. Future versions could send the same measurements to a cloud database or mobile application.

However, adding more sensors would increase memory use, wiring complexity, power consumption, and the possibility of communication conflicts. A more

scalable design should use modular driver files, a printed circuit board, structured communication protocols, and a more powerful controller such as an ESP32.

10.8 Overall Constraint Evaluation

The final prototype successfully balanced cost, functionality, and implementation time. Low-cost sensors and simple construction materials made the system affordable, while filtering, averaging, hysteresis, and error checking improved operational stability.

The main limitations were the temporary breadboard wiring, USB-based power supply, limited independent calibration, mechanical construction tolerances, and lack of a waterproof enclosure. These constraints were acceptable for an educational prototype but would need to be addressed before outdoor or long-term deployment.

11. Conclusion and Future Work

11.1 Conclusion

The Smart Weather Station was successfully designed and implemented using an Arduino Mega 2560. The system integrated several sensing modalities required by the project, including temperature, humidity, light intensity, atmospheric pressure, air-quality indication, weight measurement, and water-flow measurement.

The DHT22 was used to measure ambient temperature and relative humidity. The BH1750 measured light intensity and controlled the automatic clothesline mechanism. The BMP280 provided pressure and estimated altitude readings. The MQ135 provided a relative indication of air-condition changes. The load cell and HX711 measured weight, while the flow sensor measured water-flow rate and accumulated volume.

The system also included an LCD, push button, servo motor, and buzzer. The LCD displayed the measurements and operating status across multiple screens. The push button allowed the user to navigate through the screens. The servo automatically opened or closed the clothesline according to the measured light

level, and the buzzer produced an intermittent warning when the filtered temperature exceeded 30°C.

The firmware incorporated filtering, averaging, error checking, button debounce, servo hysteresis, pressure-trend analysis, minimum and maximum temperature tracking, bright-time accumulation, weather-condition classification, and CSV data generation.

The load-cell and water-flow sensors were manually calibrated using final factors of -400000 and 7.6, respectively. The remaining sensors were used with their digital output, manufacturer compensation, or relative readings, without additional project-specific calibration factors.

Experimental error analysis showed that the load-cell and flow-sensor measurements could provide acceptable performance for a prototype when stable mechanical and environmental conditions were maintained. Measurement uncertainty remained affected by vibration, sensor positioning, water pressure, airflow, light orientation, power-supply variation, and limited reference equipment.

Several technical problems were encountered during development. The original rain-sensor approach was replaced because its readings were unreliable for the intended control logic. The BH1750 light sensor was used instead to control the automatic clothesline. SD-card logging was also removed after repeated integration problems, and CSV data was transmitted through the Serial interface for collection on a computer.

Overall, the project achieved its main objective of creating an integrated environmental-monitoring and automatic-response system. It demonstrated sensor interfacing, signal processing, calibration, actuator control, real-time visualization, data logging, mechanical construction, and teamwork. The final system was suitable as an educational Smart Weather Station prototype, although additional development would be necessary for accurate outdoor and long-term operation.

11.2 Future Work

A future version of the project could include a properly installed rain sensor with improved shielding and calibration. This would allow the clothesline to close immediately when rain is detected rather than relying mainly on light and forecast conditions.

Wind-speed and wind-direction sensors could be added to provide more complete weather information. An ultraviolet sensor could also be included to estimate sun exposure, while a soil-moisture sensor could extend the system for garden or greenhouse use.

The Arduino Mega could be replaced or supported by an ESP32 to add Wi-Fi and Bluetooth connectivity. Measurements could then be uploaded to a cloud platform and displayed through a mobile application or web dashboard. The user could receive notifications for high temperature, poor air conditions, rainfall, abnormal pressure changes, or water-flow events.

A real-time clock module could be added so that each CSV measurement contains the actual date and time rather than only the elapsed time after startup. Local storage could be restored using a better-tested SD-card module or a different data-logging method.

The calibration process could be improved by using certified reference instruments for temperature, humidity, light, and pressure. More calibration points and repeated trials would allow regression analysis, standard deviation, confidence intervals, and more accurate uncertainty calculations.

The MQ135 could be replaced with a more selective air-quality sensor capable of reporting calibrated quantities such as carbon dioxide, volatile organic compounds, or particulate matter. This would make the air-quality results more meaningful than the current raw analog classification.

The mechanical structure could be redesigned using a weather-resistant enclosure. The electronic components should be separated from the water-flow system, and waterproof connectors should be used. The load-cell platform could be strengthened and mechanically isolated from vibration.

A dedicated regulated power supply could be added for the servo and MQ135. A battery and solar charging system could make the station portable and suitable for outdoor operation. Power-saving modes could also be used to extend battery life.

The weather-forecast function could be improved by collecting data over several hours or days rather than using only short-term local rules. More advanced prediction could use historical data, statistical models, or machine-learning methods.

These improvements would transform the current educational prototype into a more accurate, connected, reliable, and scalable environmental-monitoring system.

12. References

- [1] Arduino, “Arduino Mega 2560 Rev3,” Arduino Hardware Documentation.
- [2] Arduino, “Arduino Language Reference,” Arduino Documentation.
- [3] Aosong Electronics Co., Ltd., “DHT22 Digital Temperature and Humidity Sensor Datasheet.”
- [4] ROHM Semiconductor, “BH1750FVI Digital 16-bit Serial Output Type Ambient Light Sensor IC Datasheet.”
- [5] Bosch Sensortec, “BMP280 Digital Pressure Sensor Datasheet.”
- [6] Zhengzhou Winsen Electronics Technology Co., Ltd., “MQ135 Semiconductor Sensor for Air Quality Datasheet.”
- [7] Avia Semiconductor, “HX711 24-Bit Analog-to-Digital Converter for Weigh Scales Datasheet.”
- [8] “YF-S201 Water Flow Sensor Technical Datasheet.”
- [9] Arduino, “Servo Library Documentation.”
- [10] Arduino, “Wire Library Documentation.”
- [11] Adafruit Industries, “Adafruit BMP280 Library Documentation.”
- [12] Adafruit Industries, “DHT Sensor Library Documentation.”
- [13] Christopher Laws, “BH1750 Arduino Library Documentation.”
- [14] Bogde, “HX711 Arduino Library Documentation.”
- [15] Frank de Brabander, “LiquidCrystal I2C Arduino Library Documentation.”

[16] Lebanese University, Faculty of Engineering III, Department of Computer and Communications Engineering, “Sensors and Instrumentation Final Project Proposal,” Academic Year 2025–2026.

[17] Arduino, “Using millis() for Non-Blocking Timing,” Arduino Documentation.

[18] Microsoft, “Create Charts and Analyze Data Using Microsoft Excel,” Microsoft Support Documentation.